

Project : Suricata IDS Integration with Wazuh

NAME: Bijoy Sarker

Description:

This lab is done in Vmware. There are three VM Kali Linux, Parrot OS and Windows.

Table of Contents

1. Suricata Installation & Configuration
 - 1.1 Objective and Introduction
 - 1.2 Suricata Installation
 - 1.3 Identify Network Interface
 - 1.4 Configure Suricata
 - 1.5 Start Suricata
 - 1.6 Generate Traffic
 - 1.7 Check Alerts
2. Suricata Rule Writing
 - 2.1 Update Ruleset
 - 2.2 Create Custom Rules File
 - 2.3 Copy Rules and Verify Them
 - 2.4 Enable local.rules
 - 2.5 Reload Rules (No Restart)
 - 2.6 Test Rules
 - 2.7 Verify Alerts
3. Suricata + Wazuh Integration
 - 3.1 Configure Wazuh Agent
 - 3.2 Restart Agent / Manager
 - 3.3 Verify Integration
 - 3.4 Check Wazuh Dashboard
 - 3.5 Correlation Exercise: Suricata + Wazuh
 - 3.6 Event Flow Understanding

4. GeoIP Blocking & Firewall
 - 4.1 Install Required Tools
 - 4.2 Download Country Blocklist
 - 4.3 Create IP Set
 - 4.4 Add IPs to Set
 - 4.5 Apply Firewall Rule
 - 4.6 Verify Rules
 - 4.7 Test Blocking
 - 4.8 Key Concepts
5. Conclusion

1. Suricata Installation & Configuration

1.1 Introduction and Objective

This report documents the activities completed for SEIM Lab on Suricata and Wazuh. The main focus of this week was to expand the existing SOC lab by adding network-based detection and basic network-level blocking controls.

In the previous weeks, the lab environment mainly relied on Wazuh for host-based monitoring. Wazuh is useful for detecting events such as logins, file modifications, and process execution on monitored machines. However, host-based tools alone cannot fully show what is happening across the network. For example, if an attacker performs reconnaissance, port scanning, or suspicious web requests, those activities may be visible on the network before they produce host-level evidence.

To address this gap, Suricata was introduced as a network intrusion detection system (IDS). Suricata inspects live network traffic and generates alerts when it detects known attack patterns or suspicious behavior. By integrating Suricata with Wazuh, both host-based and network-based events can be viewed together in a single monitoring workflow.

This report is written in a step-by-step style so that a beginner can understand not only what commands were used, but also why each configuration step was necessary. Where possible, the purpose of each tool and command is explained in simple terms.

The objective of this task was to install and configure Suricata IDS in the lab environment and verify that it could detect live network traffic. Suricata fills the network-visibility gap that Wazuh does not cover by default, while Wazuh continues to monitor host-based events.

In my lab, I used:

- **Kali Linux** as attacker machine and Suricata sensor
- **Parrot OS** as Wazuh Manager/Dashboard and additional Suricata sensor
- **Windows 10** as monitored endpoint

The training material describes Suricata as the network inspection layer that outputs alerts through `fast.log` and structured telemetry through `eve.json`, which is then forwarded into Wazuh. The architecture diagram in the training slides shows this exact flow: raw network traffic → Suricata engine → `fast.log` / `eve.json` → Wazuh agent ingestion → Wazuh manager.

1.2 Suricata Installation

What I did

I first updated the system packages, then installed Suricata.

Commands used on Kali

```
sudo apt update && sudo apt upgrade -y  
sudo apt install suricata -y
```

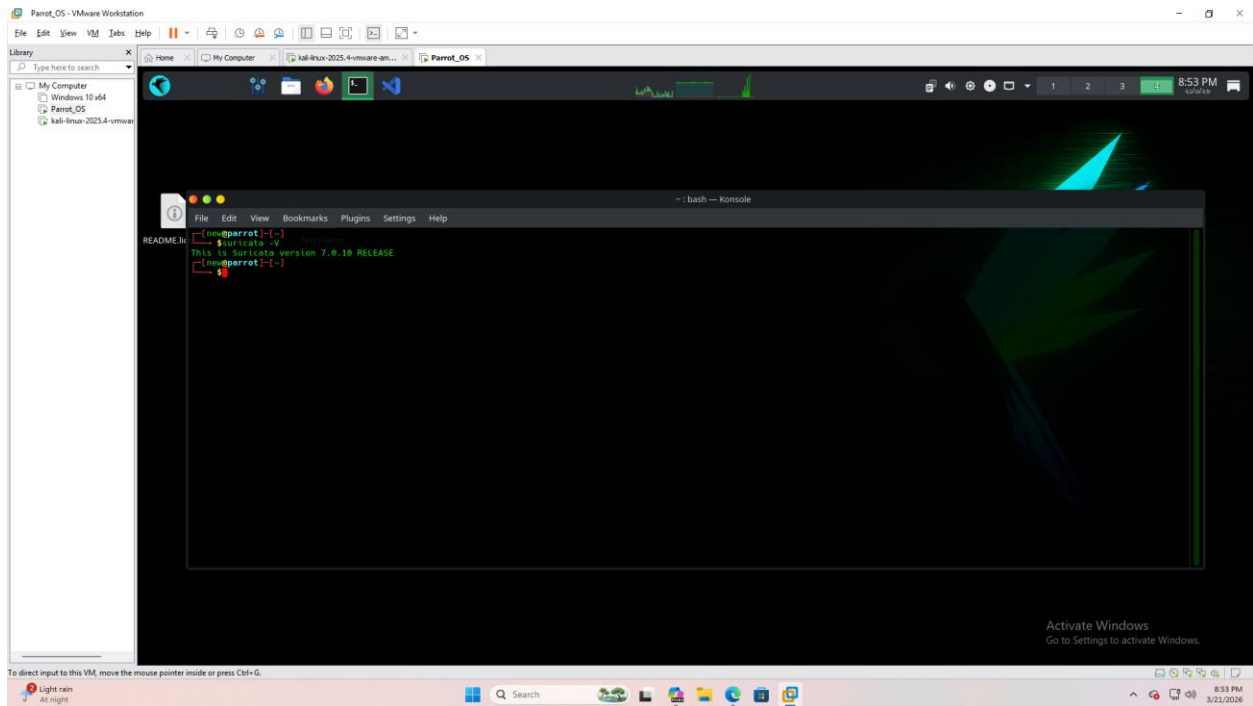
Commands used on Parrot

```
sudo apt update && sudo apt upgrade -y  
sudo apt install suricata -y
```

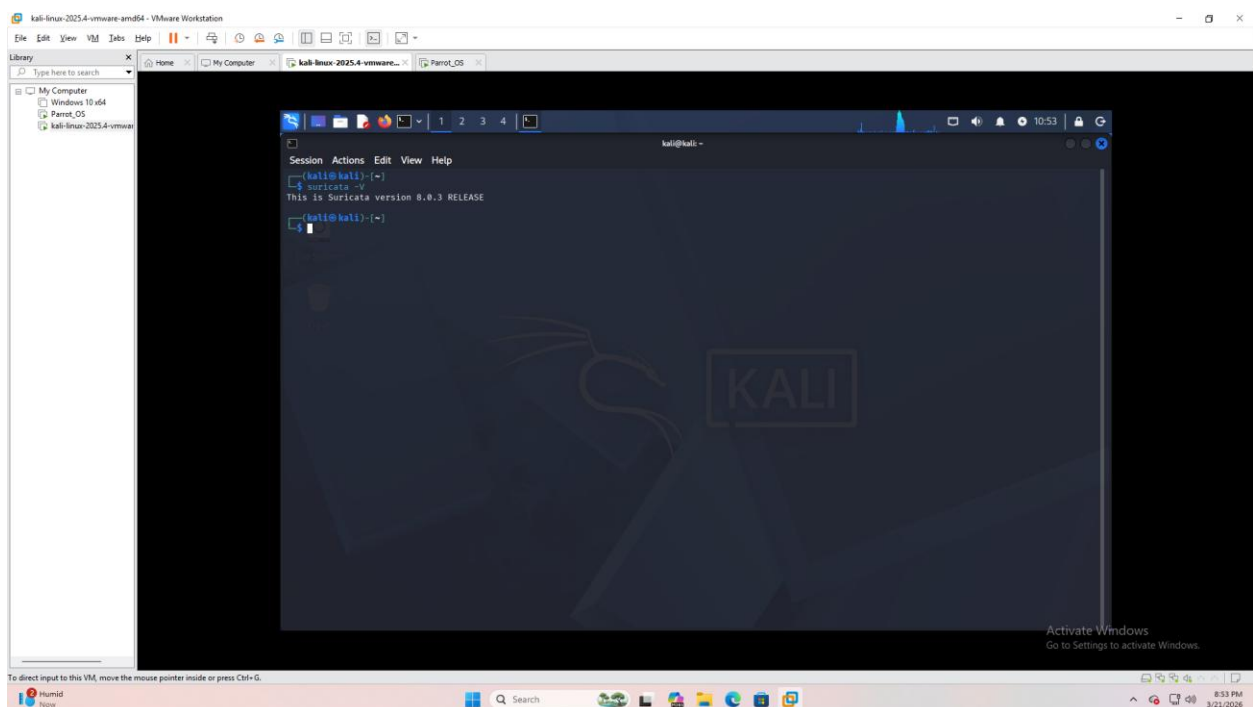
Verify installation

```
suricata -V
```

Parrot OS:



Kali Linux:



1.3 Identify Network Interface

What I did

I checked the available interfaces to identify the **adapter**. The brief says Suricata must monitor the interface that carries the lab traffic; otherwise, Suricata may start successfully but see no packets and generate no alerts.

Command used

```
ifconfig
```

My result on Parrot OS

From my system output:

- ens33 = 192.168.136.131
- ens36 = 192.168.56.137

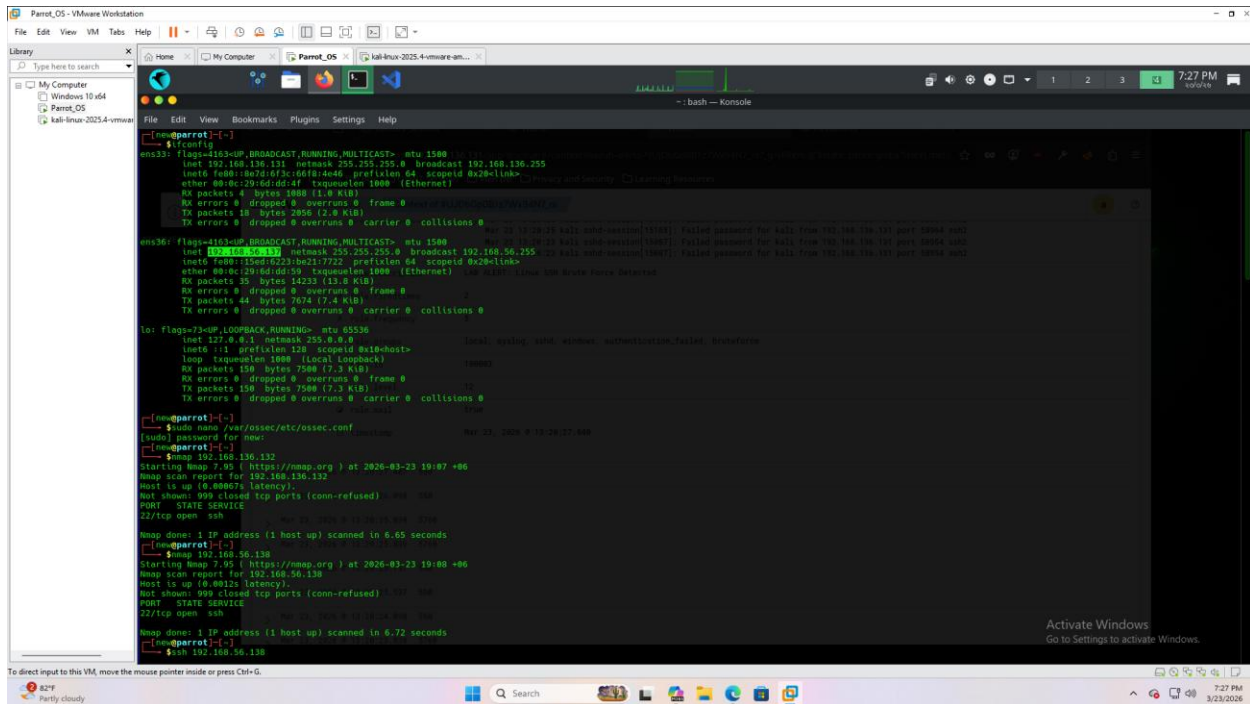
The address 192.168.136.130 appeared after adding the host-only adapter in VMware, so:

- **ens33 = host-only**
- **ens36 = NAT**

You should do the same check on Parrot and identify its host-only interface and IP.

.

Screenshot



```
[newparrot@kali:~]$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.136.131 netmask 255.255.255.0 broadcast 192.168.136.255
    inet6 fe80::672:fe2:fe48::e4d prefixlen 64 scopeid 0x20<link>
    ether 88:0c:29:16:d0:4f txqueuelen 1000 (Ethernet)
    RX packets 4 bytes 1008 (1.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18 bytes 2056 (2.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens36: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.132 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::15e:622:b21:7722 prefixlen 64 scopeid 0x20<link>
    ether 88:0c:29:16:d0:59 txqueuelen 1000 (Ethernet)
    RX packets 35 bytes 1423 (1.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 44 bytes 2674 (2.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 150 bytes 7500 (7.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 130 bytes 7500 (7.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

[newparrot@kali:~]$ sudo nano /etc/ossec/etc/ossec.conf
[ossec] password for new:
[newparrot@kali:~]$ sudo nmap 192.168.136.132
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 19:07 +06
Nmap scan report for 192.168.136.132
Host is up (0.00007s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
Nmap done: 1 IP address (1 host up) scanned in 6.65 seconds
[newparrot@kali:~]$ sudo nmap 192.168.56.138
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 19:08 +06
Nmap scan report for 192.168.56.138
Host is up (0.00125s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
Nmap done: 1 IP address (1 host up) scanned in 6.72 seconds
[newparrot@kali:~]$ sudo nmap 192.168.56.138
```

My result on Kali Linux

From my system output:

- eth0= 192.168.136.132
- eth1 = 192.168.56.138

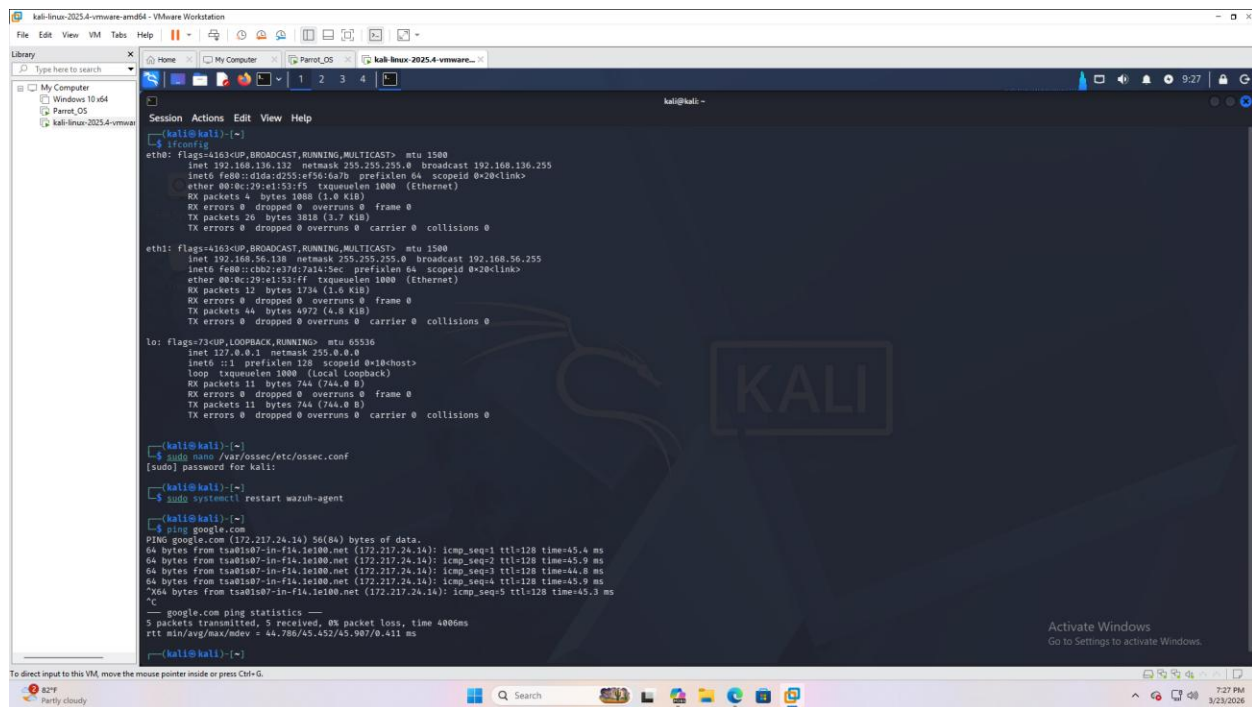
The address 192.168.136.128 appeared after adding the host-only adapter in VMware, so:

- eth0 = host-only
- eth1 = NAT

What to write

The host-only interface was selected because all lab attack traffic such as nmap, ping, and curl was performed over the isolated host-only network, not the NAT network. NAT was kept for internet access and package installation, while host-only was used for IDS monitoring

Screenshot



```
(kali@kali):~$ ifconfig
eth8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.136.132 netmask 255.255.255.0 broadcast 192.168.136.255
    inet6 fe80::d1a:d255:ef56:6a7b prefixlen 64 scopeid 0<20<link>
    ether 08:0c:29:e1:53:ff txqueuelen 1000 (Ethernet)
    RX packets 4 bytes 1008 (1.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 20 bytes 3816 (3.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.138 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::cb2:e37d:7a14:3ec prefixlen 64 scopeid 0<20<link>
    ether 00:0c:29:e1:53:ff txqueuelen 1000 (Ethernet)
    RX packets 12 bytes 1734 (1.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 44 bytes 4972 (4.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (local loopback)
    RX packets 11 bytes 744 (744.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 11 bytes 744 (744.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali):~$ sudo nano /var/ossec/etc/ossec.conf
[sudo] password for kali:
(kali@kali):~$ sudo systemctl restart wazuh-agent
(kali@kali):~$ ping google.com
PING google.com (172.217.24.14) 56(84) bytes of data:
64 bytes from tsao1s07-in-f14.1e100.net (172.217.24.14): icmp_seq=1 ttl=128 time=45.4 ms
64 bytes from tsao1s07-in-f14.1e100.net (172.217.24.14): icmp_seq=2 ttl=128 time=45.9 ms
64 bytes from tsao1s07-in-f14.1e100.net (172.217.24.14): icmp_seq=3 ttl=128 time=44.8 ms
64 bytes from tsao1s07-in-f14.1e100.net (172.217.24.14): icmp_seq=4 ttl=128 time=45.9 ms
64 bytes from tsao1s07-in-f14.1e100.net (172.217.24.14): icmp_seq=5 ttl=128 time=45.3 ms
^C
  google.com ping statistics ---
  5 packets transmitted, 5 received, 0% packet loss, time 4000ms
 rtt min/avg/max/mdev = 44.786/45.452/45.907/0.411 ms
(kali@kali):~$
```

1.4 Configure Suricata

What I did

I edited the main Suricata configuration file:

```
sudo nano /etc/suricata/suricata.yaml
```

Important settings

HOME_NET

I changed the lab subnet to match the host-only network:

```
HOME_NET: "[192.168.136.0/24]"
```

af-packet

On Kali:


```
af-packet:
- interface: ens36
  cluster-id: 99
  cluster-type: cluster_flow
  defrag: yes
  threads: auto
  use-mmap: yes
```

On Parrot, I used the host-only interface name for Parrot in the same section.

eve-log

I verified that JSON logging was enabled:

```
- eve-log:
  enabled: yes
  filetype: regular
  filename: eve.json
```

rule-files

I made sure the custom rules file was listed:

```
rule-files:
- suricata.rules
- local.rules
```

Why these settings matter

- HOME_NET defines which internal network is considered protected
- af-packet tells Suricata which Linux capture interface to inspect
- eve.json is needed for Wazuh integration
- local.rules is required for custom

The training slide named “Core Configuration (suricata.yaml)” highlights these same areas: AF-PACKET, VARS, EVE-LOG, and RULE-FILES.

Config validation

After editing the file, I tested it:

On Parrot OS:

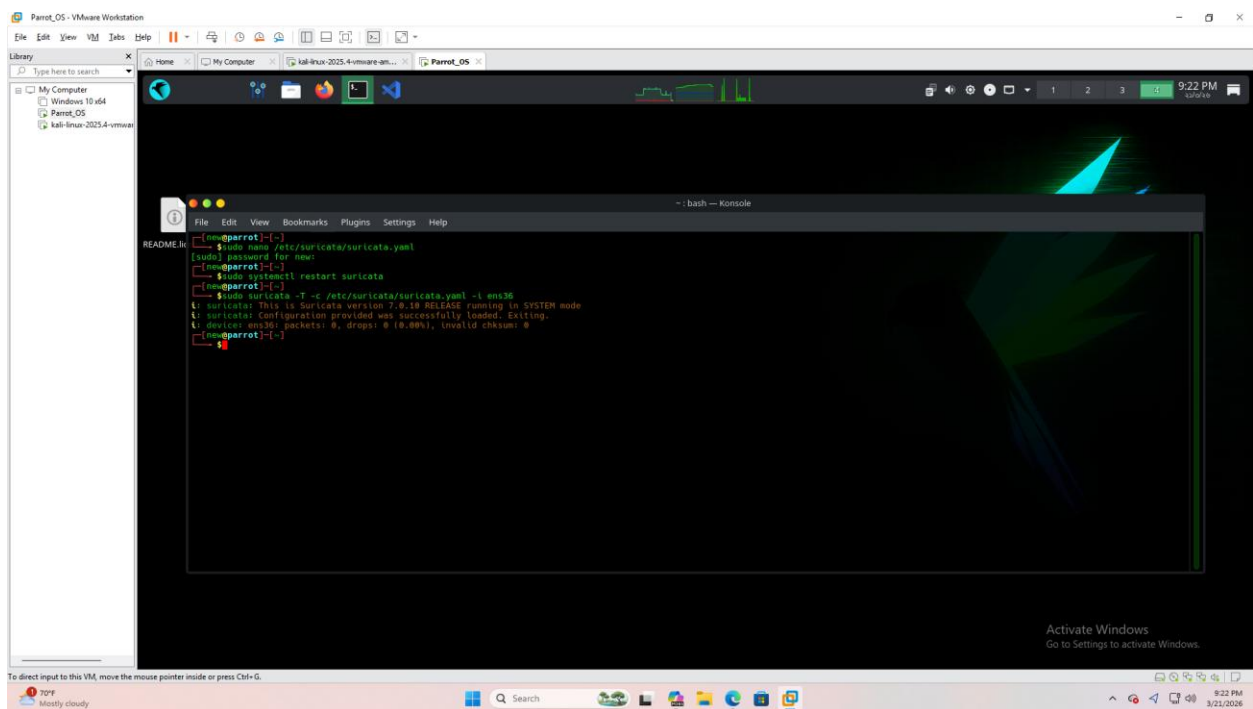
```
sudo suricata -T -c /etc/suricata/suricata.yaml -i ens36
```

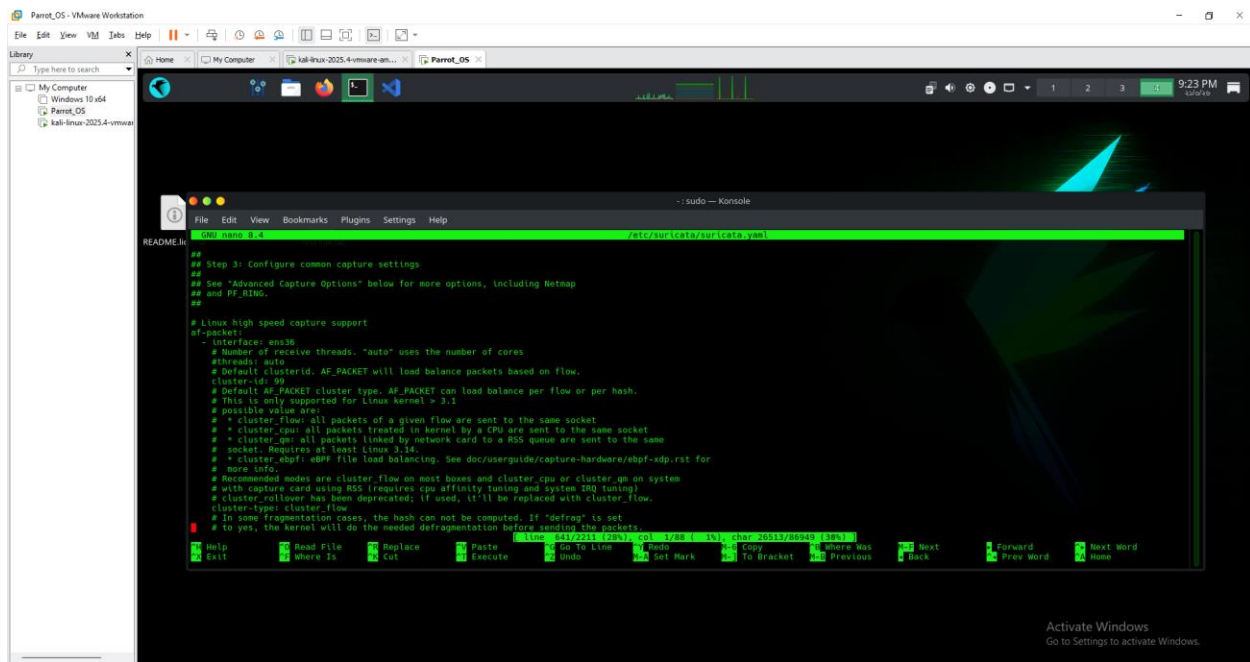
On Kali Linux:

```
sudo suricata -T -c /etc/suricata/suricata.yaml -i eth1
```

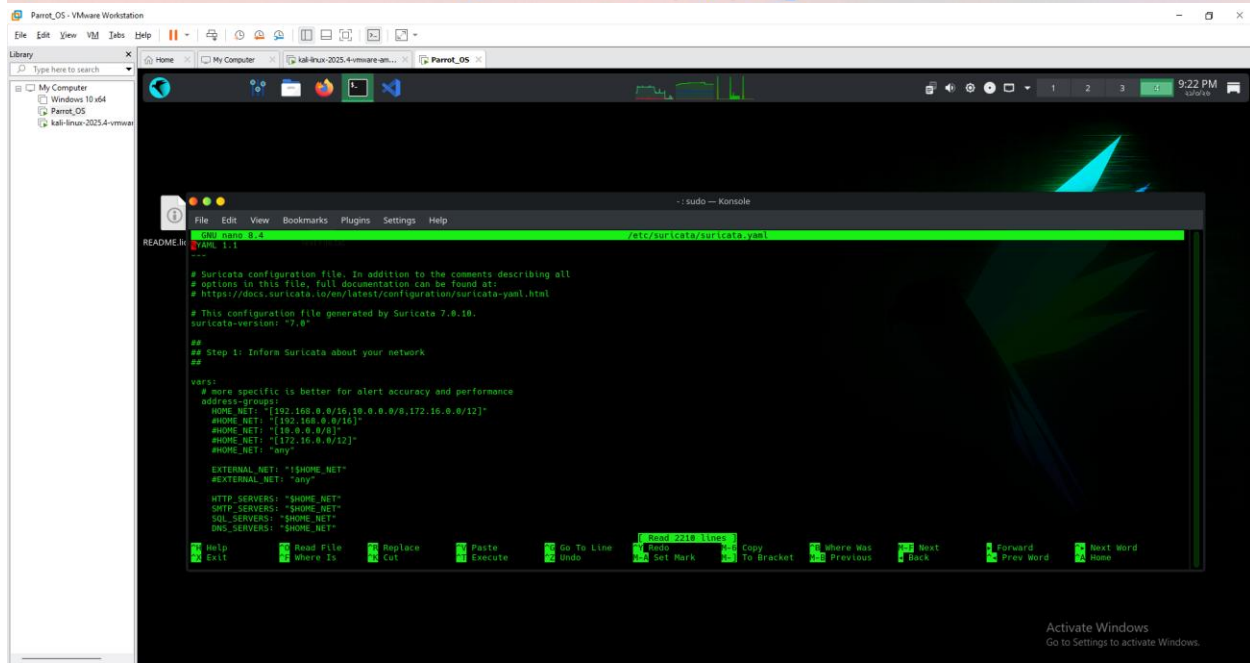
Screenshots

Parrot OS:

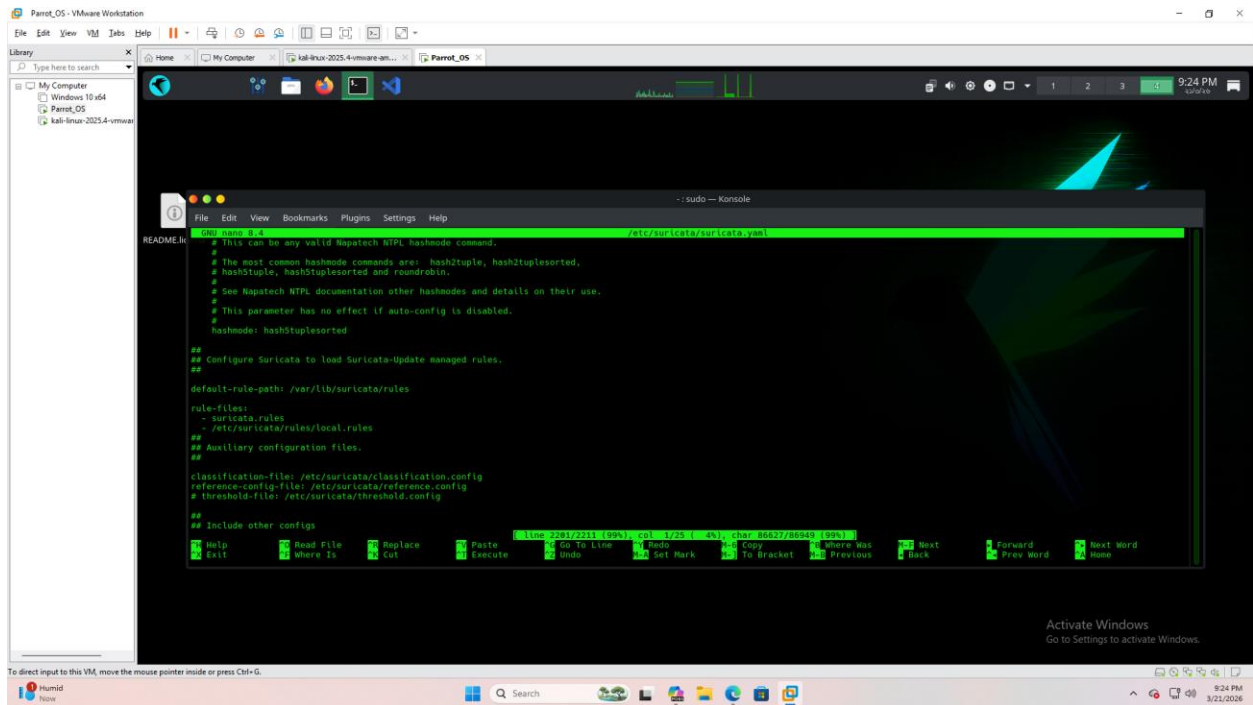




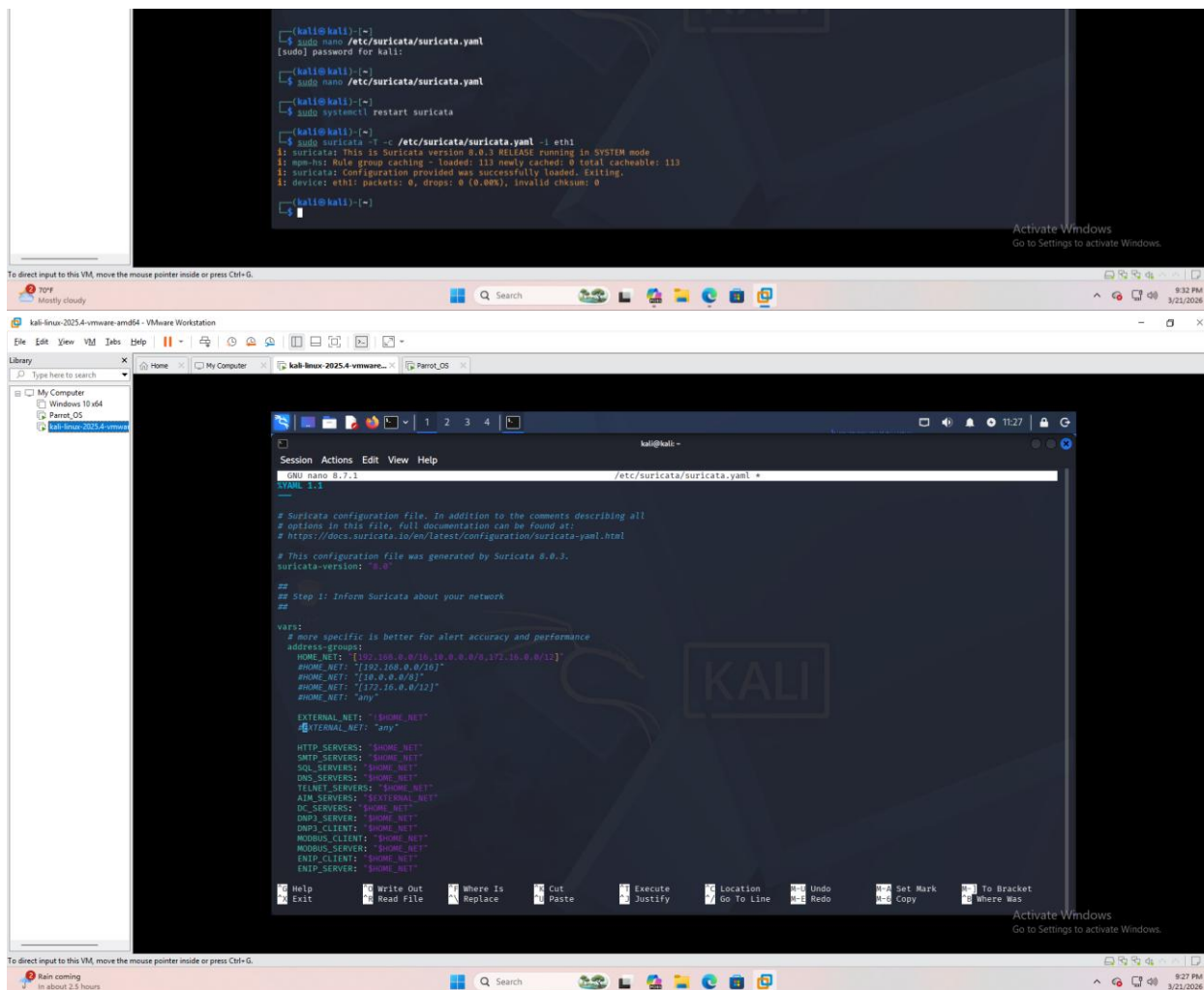
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.



Kali Linux:




```
sudo systemctl start suricata
sudo systemctl enable suricata
sudo systemctl status suricata
```

What I checked

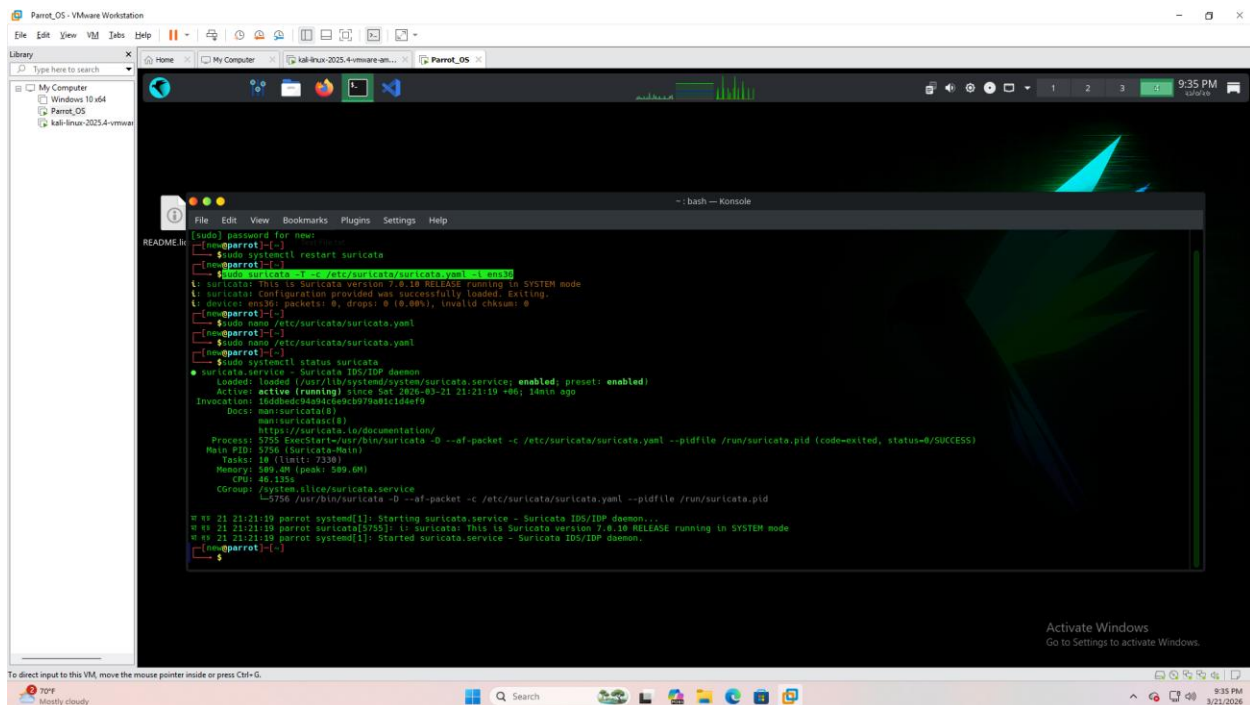
The service had to show:

- loaded
- active (running)

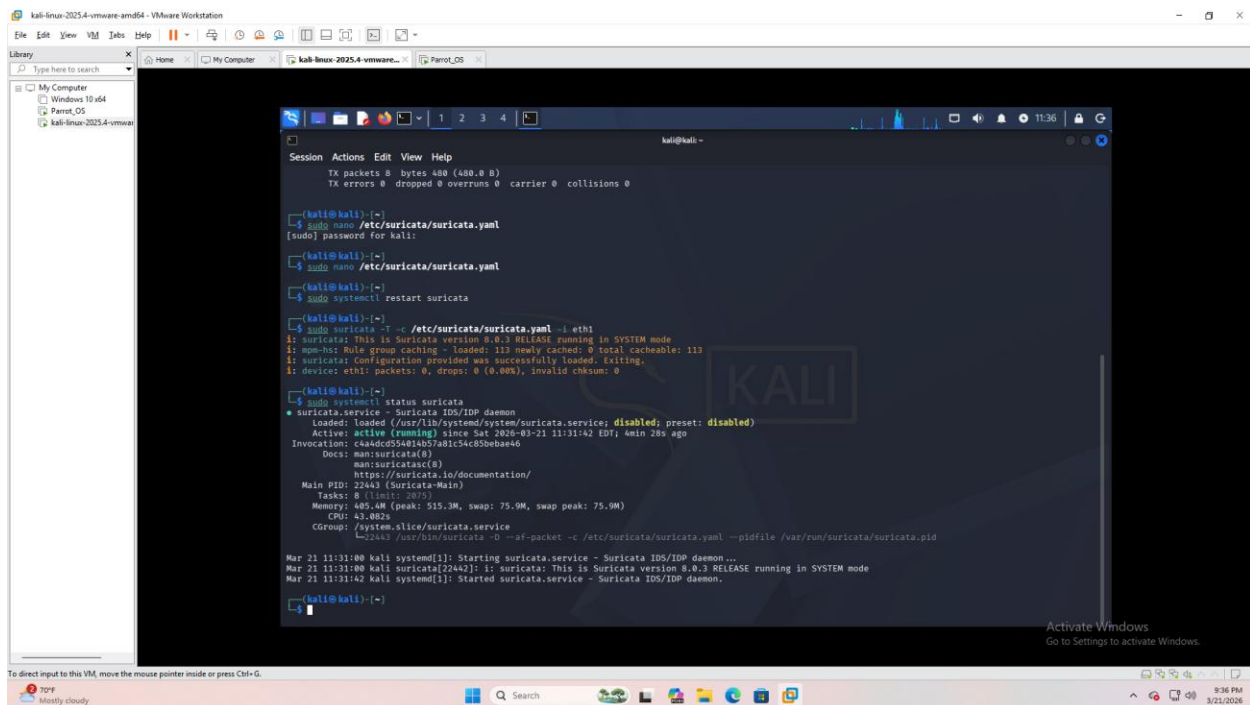
This proves the service is running, although the brief notes that this alone does not prove it is seeing the correct traffic. That must be verified with log output after traffic generation.

Screenshots

Parrot OS:



Kali Linux:



1.6 Generate Traffic

What I did

To test Suricata detection, I generated traffic from Kali to other lab systems over the NAT network.

Commands used

ping 192.168.56.137

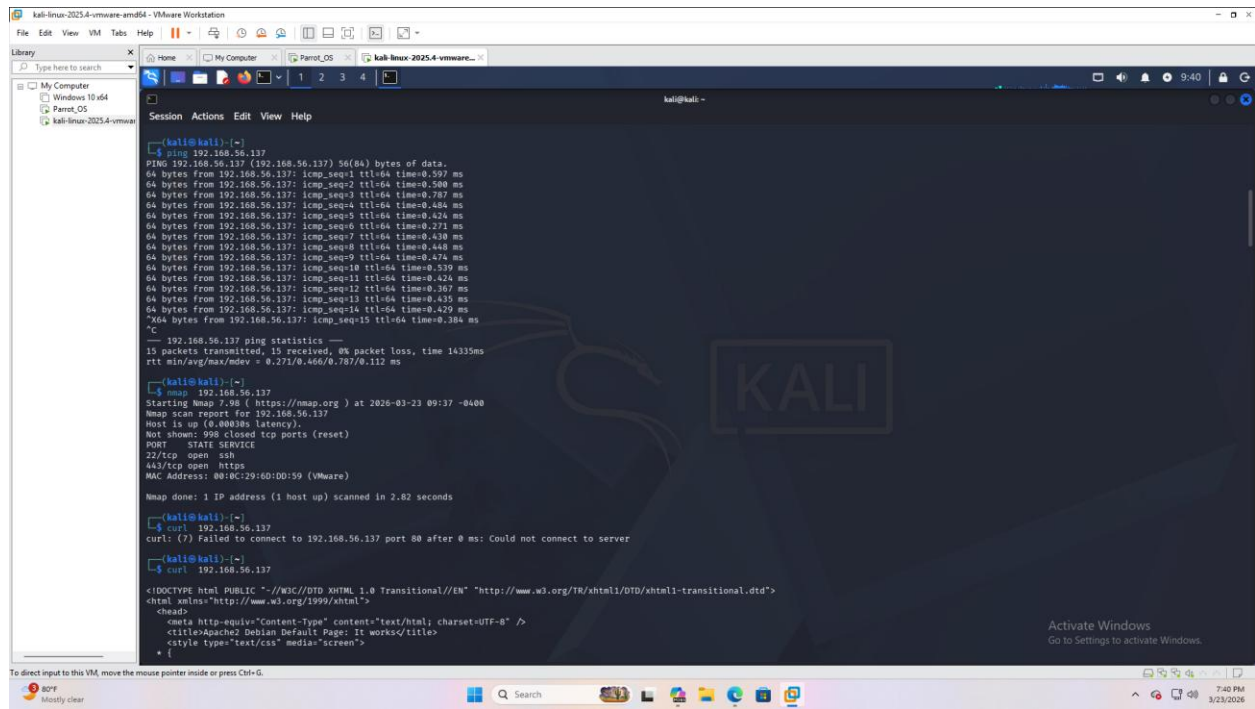
nmap 192.168.56.137

Curl 192.168.56.137

Important note

For Suricata testing, I used NAT IPs, because the NAT interface was the one Suricata was monitoring.

Screenshot:



The screenshot shows a Kali Linux terminal window with the following commands and output:

```
kali@kali:~$ ping 192.168.56.137
PING 192.168.56.137 (192.168.56.137) 56(84) bytes of data:
64 bytes from 192.168.56.137: icmp_seq=1 ttl=64 time=0.597 ms
64 bytes from 192.168.56.137: icmp_seq=2 ttl=64 time=0.580 ms
64 bytes from 192.168.56.137: icmp_seq=3 ttl=64 time=0.787 ms
64 bytes from 192.168.56.137: icmp_seq=4 ttl=64 time=0.484 ms
64 bytes from 192.168.56.137: icmp_seq=5 ttl=64 time=0.424 ms
64 bytes from 192.168.56.137: icmp_seq=6 ttl=64 time=0.221 ms
64 bytes from 192.168.56.137: icmp_seq=7 ttl=64 time=0.430 ms
64 bytes from 192.168.56.137: icmp_seq=8 ttl=64 time=0.448 ms
64 bytes from 192.168.56.137: icmp_seq=9 ttl=64 time=0.514 ms
64 bytes from 192.168.56.137: icmp_seq=10 ttl=64 time=0.539 ms
64 bytes from 192.168.56.137: icmp_seq=11 ttl=64 time=0.424 ms
64 bytes from 192.168.56.137: icmp_seq=12 ttl=64 time=0.367 ms
64 bytes from 192.168.56.137: icmp_seq=13 ttl=64 time=0.435 ms
64 bytes from 192.168.56.137: icmp_seq=14 ttl=64 time=0.409 ms
64 bytes from 192.168.56.137: icmp_seq=15 ttl=64 time=0.384 ms
^C
--- 192.168.56.137 ping statistics ---
15 packets transmitted, 15 received, 0% packet loss, time 1433ms
rtt min/avg/max/mdev = 0.271/0.466/0.787/0.112 ms

kali@kali:~$ nmap 192.168.56.137
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 09:37 -0400
Nmap scan report for 192.168.56.137
Host is up (0.00038s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
443/tcp   open  https
MAC Address: 08:0C:29:6D:DD:59 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.82 seconds

kali@kali:~$ curl 192.168.56.137
curl: (7) Failed to connect to 192.168.56.137 port 80 after 0 ms: Could not connect to server

kali@kali:~$ curl http://192.168.56.137
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Debian Default Page: It works</title>
    <style type="text/css" media="screen">
      {
```

What I did

To test Suricata detection, I generated traffic from Parrot OS to Kali linux systems over the NAT network.

Commands used

ping 192.168.56.138

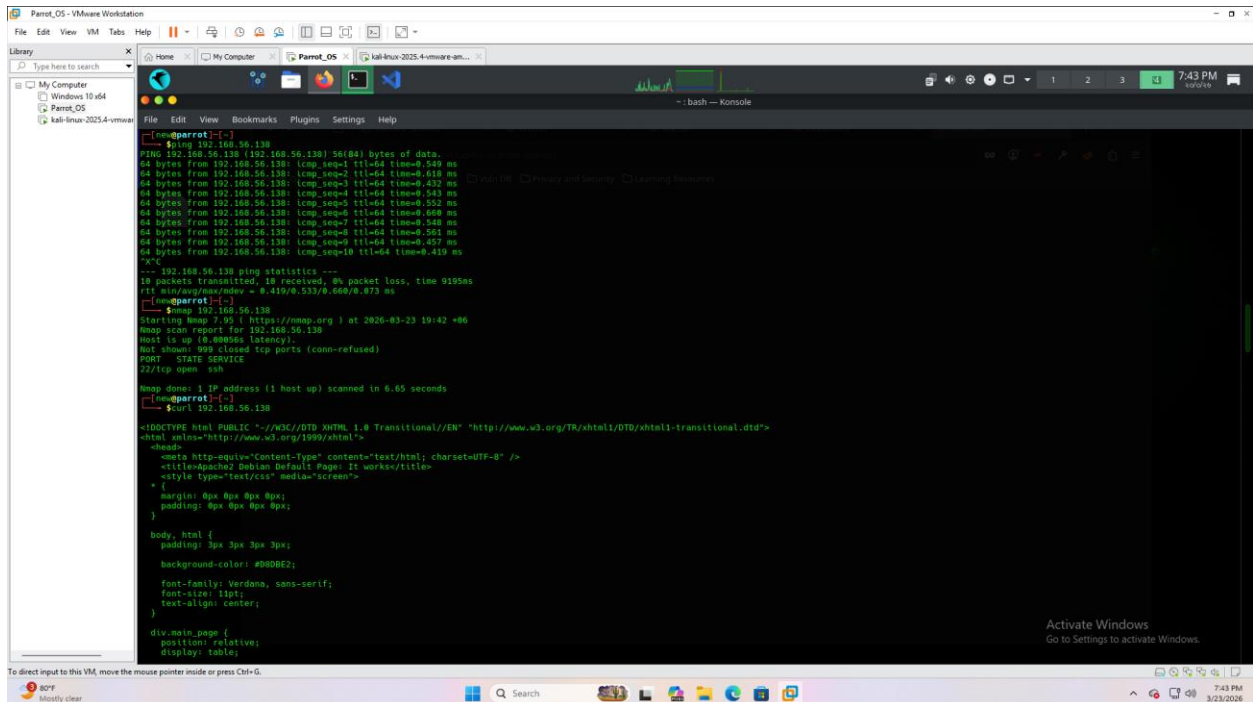
nmap 192.168.56.138

Curl http://192.168.56.138

Important note

For Suricata testing, I used NAT IPs, because the NAT interface was the one Suricata was monitoring.

Screenshot:



The screenshot shows a Parrot OS - VMware Workstation window with a terminal running several commands. The terminal output includes:

```
[root@parrot:~]# ping 192.168.56.138
PING 192.168.56.138 (192.168.56.138) 56(84) bytes of data:
64 bytes from 192.168.56.138: icmp_seq=1 ttl=64 time=0.549 ms
64 bytes from 192.168.56.138: icmp_seq=2 ttl=64 time=0.618 ms
64 bytes from 192.168.56.138: icmp_seq=3 ttl=64 time=0.432 ms
64 bytes from 192.168.56.138: icmp_seq=4 ttl=64 time=0.543 ms
64 bytes from 192.168.56.138: icmp_seq=5 ttl=64 time=0.552 ms
64 bytes from 192.168.56.138: icmp_seq=6 ttl=64 time=0.668 ms
64 bytes from 192.168.56.138: icmp_seq=7 ttl=64 time=0.548 ms
64 bytes from 192.168.56.138: icmp_seq=8 ttl=64 time=0.561 ms
64 bytes from 192.168.56.138: icmp_seq=9 ttl=64 time=0.457 ms
64 bytes from 192.168.56.138: icmp_seq=10 ttl=64 time=0.419 ms
^C
--- 192.168.56.138 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 919ms
rtt min/avg/max/mdev = 0.419/0.533/0.668/0.073 ms

[root@parrot:~]# nmap 192.168.56.138
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-23 19:42 +06
Nmap scan report for 192.168.56.138
Host is up (0.000066s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh

Nmap done: 1 IP address (1 host up) scanned in 6.65 seconds

[root@parrot:~]# curl 192.168.56.138
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Apache2 Debian Default Page: It works</title>
<style type="text/css" media="screen">
* {
margin: 0px 0px 0px 0px;
padding: 0px 0px 0px 0px;
}
body, html {
padding: 3px 3px 3px 3px;
background-color: #B8DBE2;
font-family: Verdana, sans-serif;
font-size: 10pt;
text-align: center;
}
div.main_page {
position: relative;
display: table;

```

1.7 Check Alerts

What I did

I checked both Suricata logs:

Human-readable alert log

```
sudo tail -f /var/log/suricata/fast.log
```

Structured JSON log

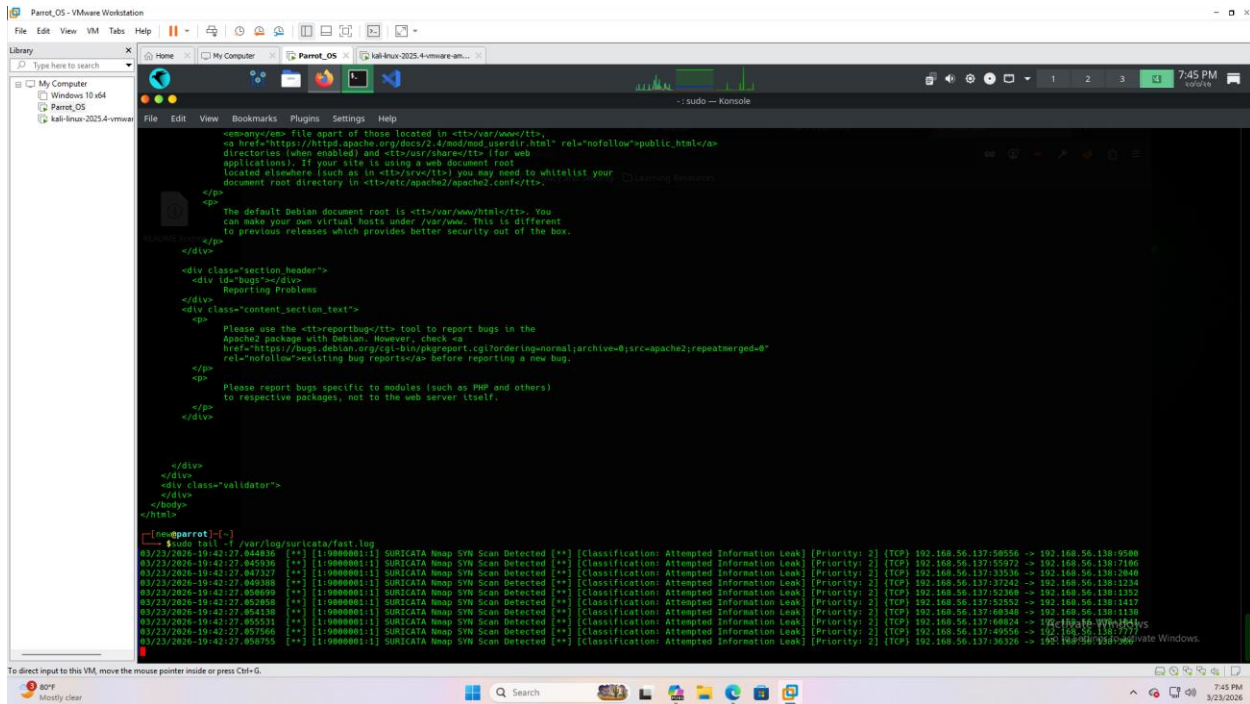
```
sudo tail -f /var/log/suricata/eve.json
```

What I observed

- fast.log showed compact alert lines
- eve.json showed structured event data in JSON format

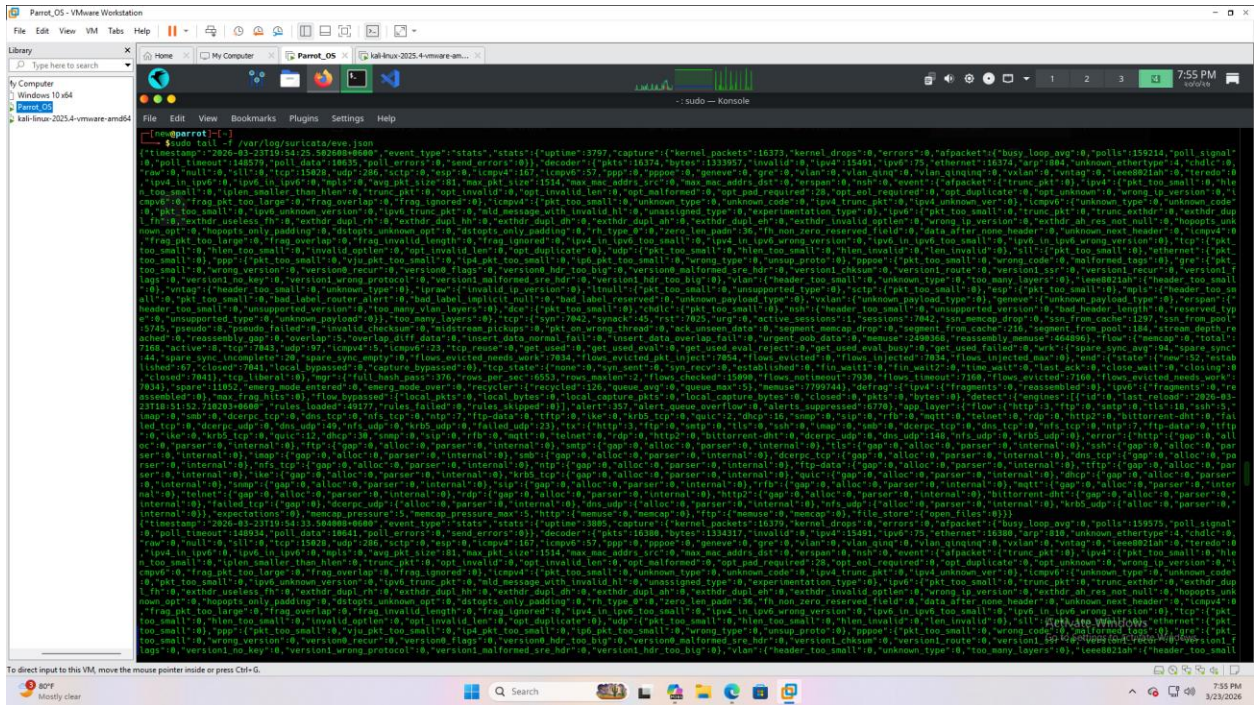
ScreenShot:

Parrot OS:

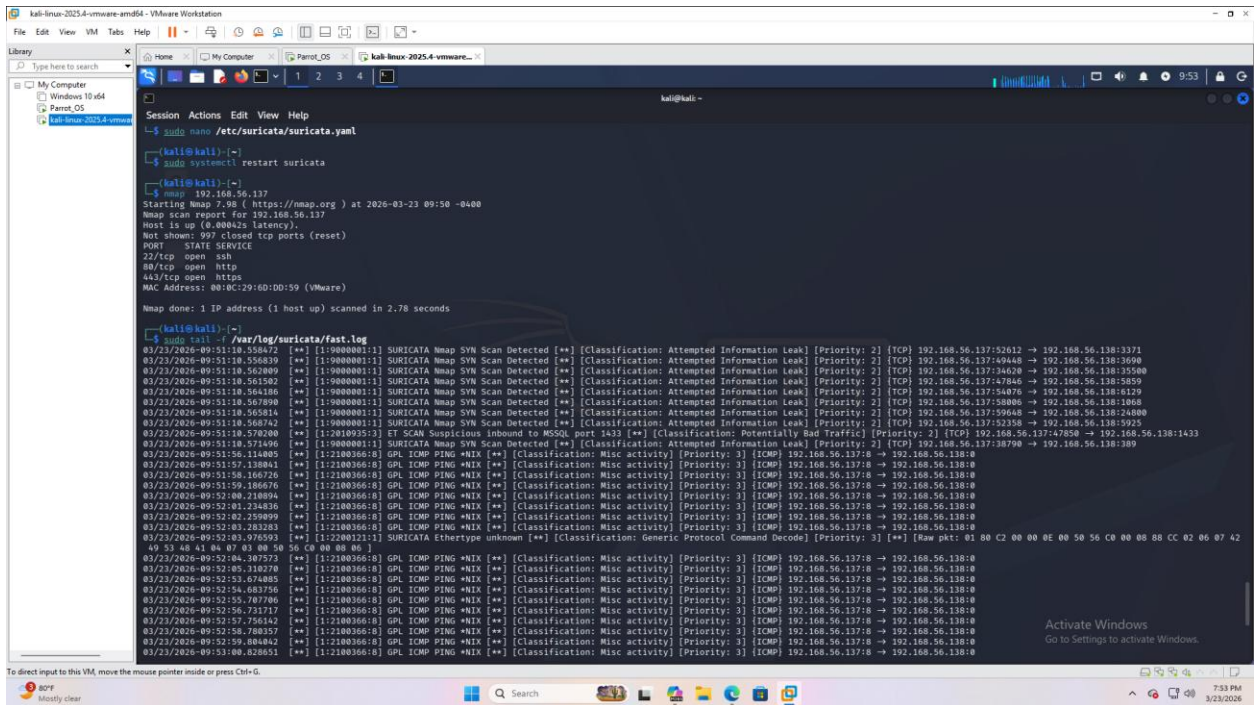


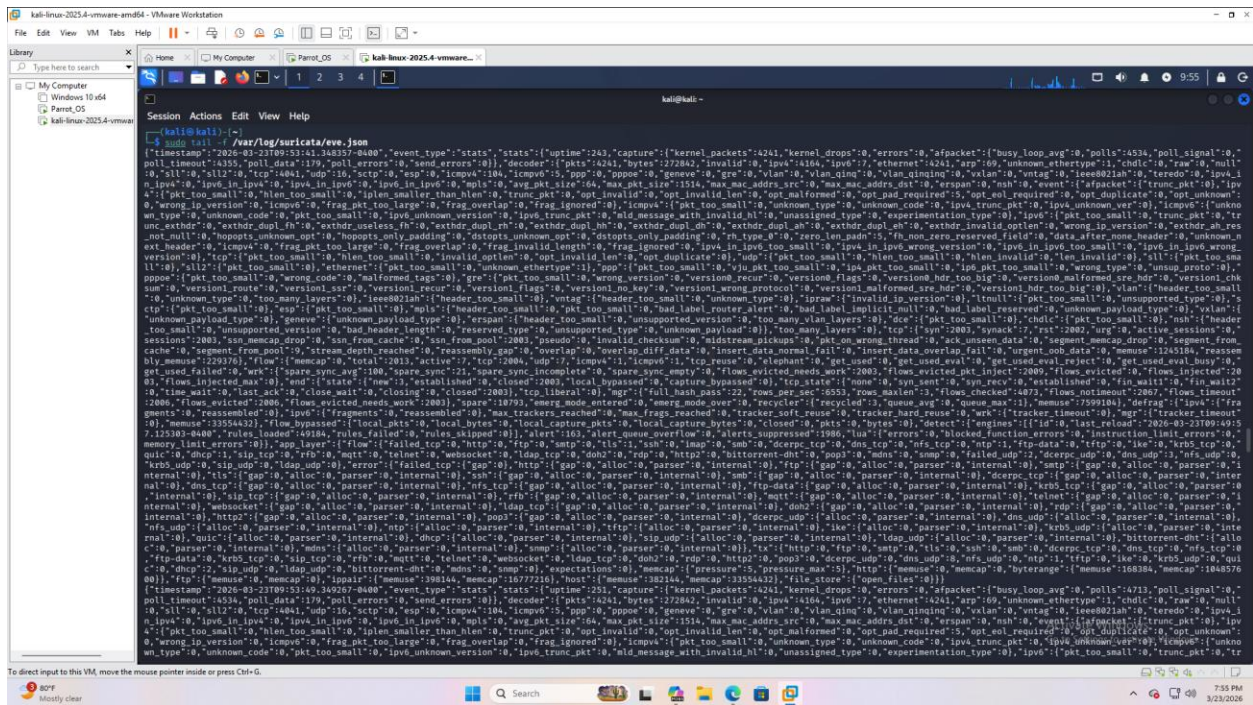
The screenshot shows a Parrot OS virtual machine running on VMware Workstation. The main window displays a web browser with a security advisory page. The terminal window shows the output of the command `sudo tail -f /var/log/suricata/fast.log`, displaying a series of Suricata alerts for attempted information leaks.

```
[root@parrot:~]# sudo tail -f /var/log/suricata/fast.log
63/23/2026-19:42:27.844836 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:58556 -> 192.168.56.138:8080
63/23/2026-19:42:27.845036 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:55972 -> 192.168.56.138:7106
63/23/2026-19:42:27.847227 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:33536 -> 192.168.56.138:2040
63/23/2026-19:42:27.849388 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:37242 -> 192.168.56.138:1234
63/23/2026-19:42:27.850899 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:52360 -> 192.168.56.138:1352
63/23/2026-19:42:27.852058 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:52352 -> 192.168.56.138:1417
63/23/2026-19:42:27.854138 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:60348 -> 192.168.56.138:1138
63/23/2026-19:42:27.855231 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:68024 -> 192.168.56.138:1135
63/23/2026-19:42:27.857866 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:40956 -> 192.168.56.138:1077
63/23/2026-19:42:27.858755 [**] [1:90000001:1] SURICATA Nmap SYN Scan Detected [**] [Classification: Attempted Information Leak] [Priority: 2] [TCP] 192.168.56.137:36326 -> 192.168.56.138:1135
```



Kali Linux:





2. Suricata Rule Writing

2.1 Update Ruleset

What I did

Before creating custom rules, I updated the default Suricata ruleset.

Commands

```
sudo suricata-update
sudo systemctl restart suricata
sudo systemctl status suricata
```

2.2 Create Custom Rules File

What I did

I created my custom rule file:

```
sudo nano /etc/suricata/rules/local.rules
```

Rules used

```
alert tcp any any -> $HOME_NET any (msg:"SURICATA Nmap SYN Scan Detected"; flags:S; threshold:type threshold, track by_src, count 20, seconds 10; classtype:attempted-recon; sid:9000001; rev:>
```

```
alert tcp any any -> $HOME_NET 80 (msg:"CRITICAL Directory Traversal"; content:"../"; priority:1; sid:9000002; rev:2;)
```

```
alert icmp any any -> $HOME_NET any (msg:"SURICATA ICMP Flood Detected"; itype:8; threshold:type threshold, track by_src, count 100, seconds 10; classtype:attempted-dos; sid:9000003; rev:1;)
```

What to write

Each rule was written to detect a specific threat scenario:

- Nmap SYN scan
- HTTP directory traversal attempt
- ICMP flood

2.3 Copy Rules and Verify Them

What I did

In the sample report, the rules were copied into `/var/lib/suricata/rules/`. If your system requires it, you can do:

```
sudo cp /etc/suricata/rules/local.rules  
/var/lib/suricata/rules/  
ls /var/lib/suricata/rules/
```

If your Suricata installation is already reading
/etc/suricata/rules/local.rules directly through suricata.yaml,
this extra copy is optional.

What to write

I verified that the custom rules file existed and was accessible to the Suricata ruleset loading process.

2.4 Enable local.rules

What I did

I edited suricata.yaml and ensured local.rules was listed in the rule-files section:

```
rule-files:  
- suricata.rules  
- local.rules
```

2.5 Reload Rules

What I did

I reloaded the rules without fully restarting the service.

```
sudo suricatasc -c reload-rules
```

Then I checked the log for errors:

```
sudo tail -f /var/log/suricata/suricata.log
```

If rule reload failed, I used:

```
sudo systemctl restart suricata
```

What to write

Rule reload was used to apply the new custom rules while minimizing interruption. Log verification confirmed that no SID conflict or syntax error prevented rule loading

2.6 Test Rules

Rule 1: Nmap SYN Scan Detection

```
nmap ip
```

Rule 2: HTTP Directory Traversal Detection

```
echo -e "GET ../../../../etc/passwd HTTP/1.1\r\nHost: ip\r\n\r\n" | nc ip 80
```

Rule 3: ICMP Flood Detection

```
sudo ping -f <ip>
```

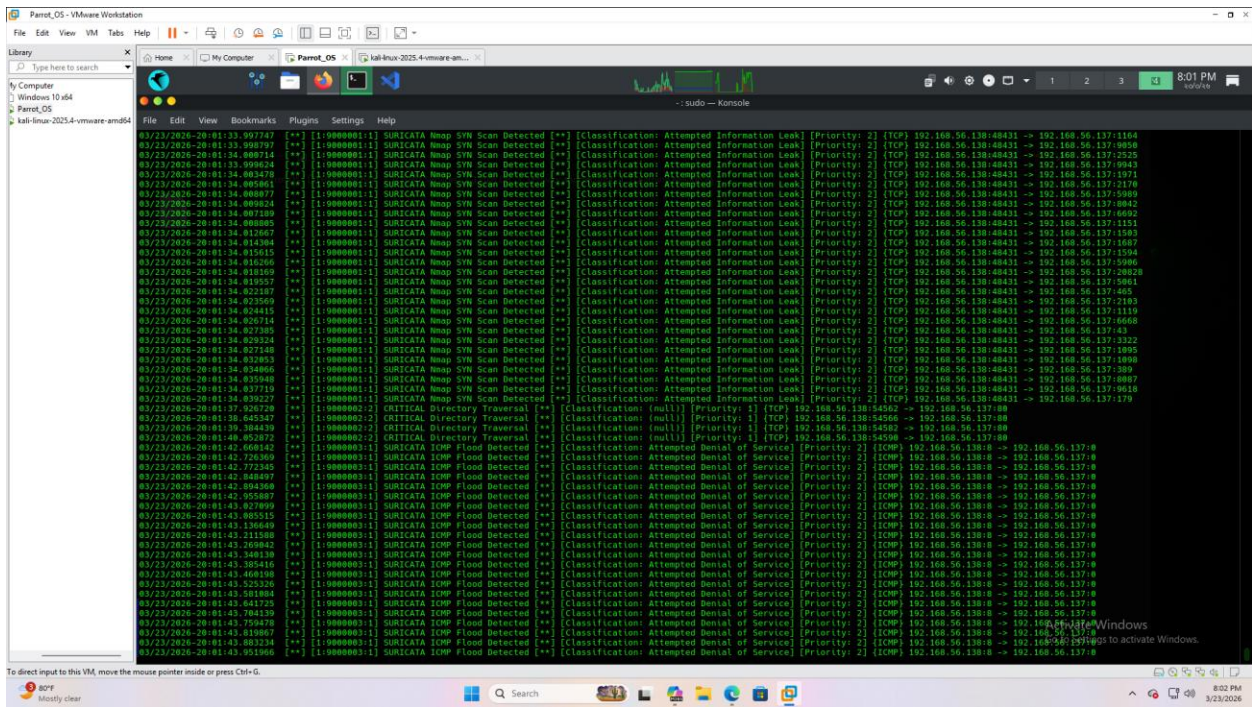
2.7 Verify Alerts

What I did

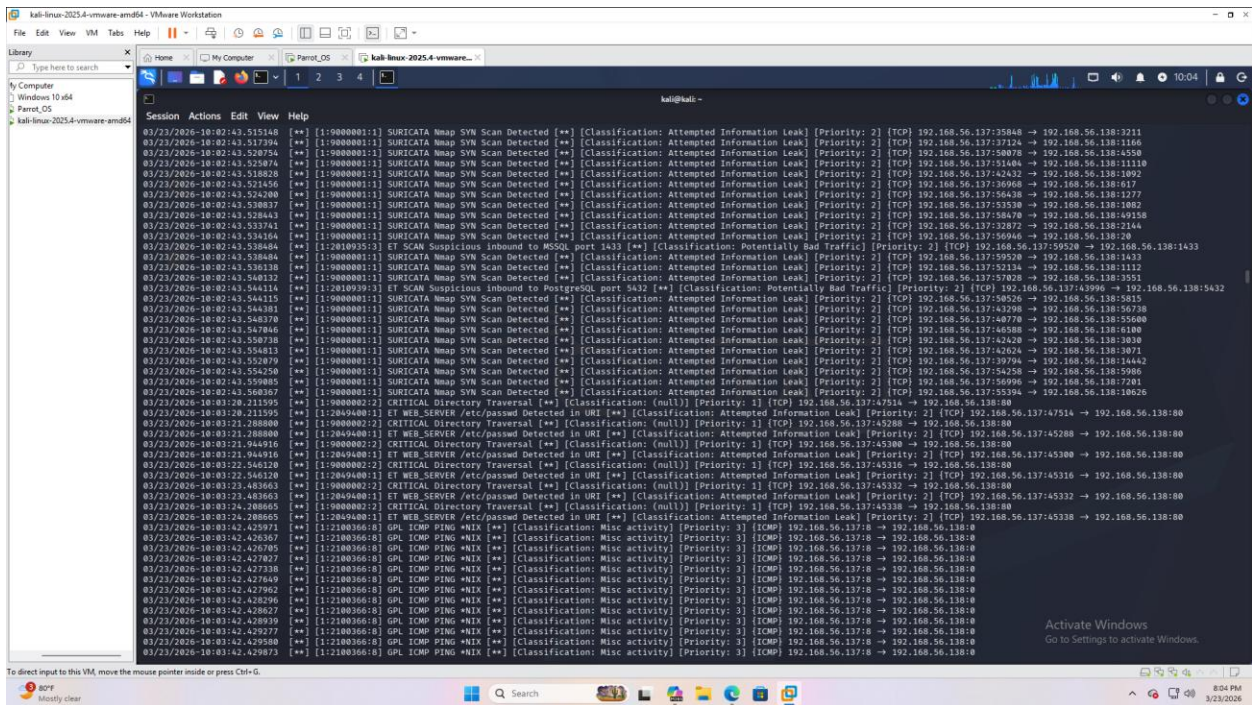
I verified the alerts in fast.log

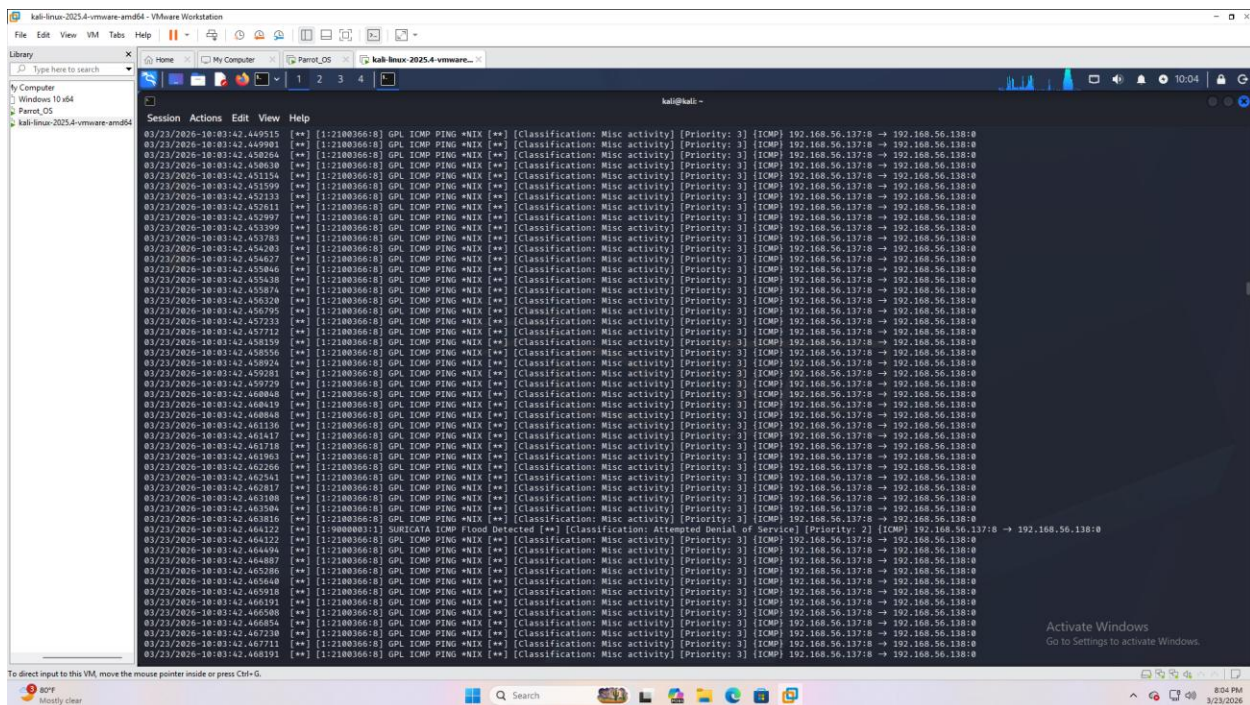
Screenshots

Parrot OS:



Kali Linux:





Important conditions

It will work **only** if these are true:

1. A web server is listening on port 80

On the target machine, port 80 must be open.

Check on the target:

```
sudo systemctl status apache2
```

If nothing is listening on port 80, Suricata may not classify it properly as HTTP traffic.

3. Suricata + Wazuh Integration

3.1 Configure Wazuh Agent

This Part describes the official integration path as: Suricata on Kali writes alerts to `eve.json`, then the **Wazuh agent on Kali** reads that file and forwards it to the Wazuh manager.

What I did

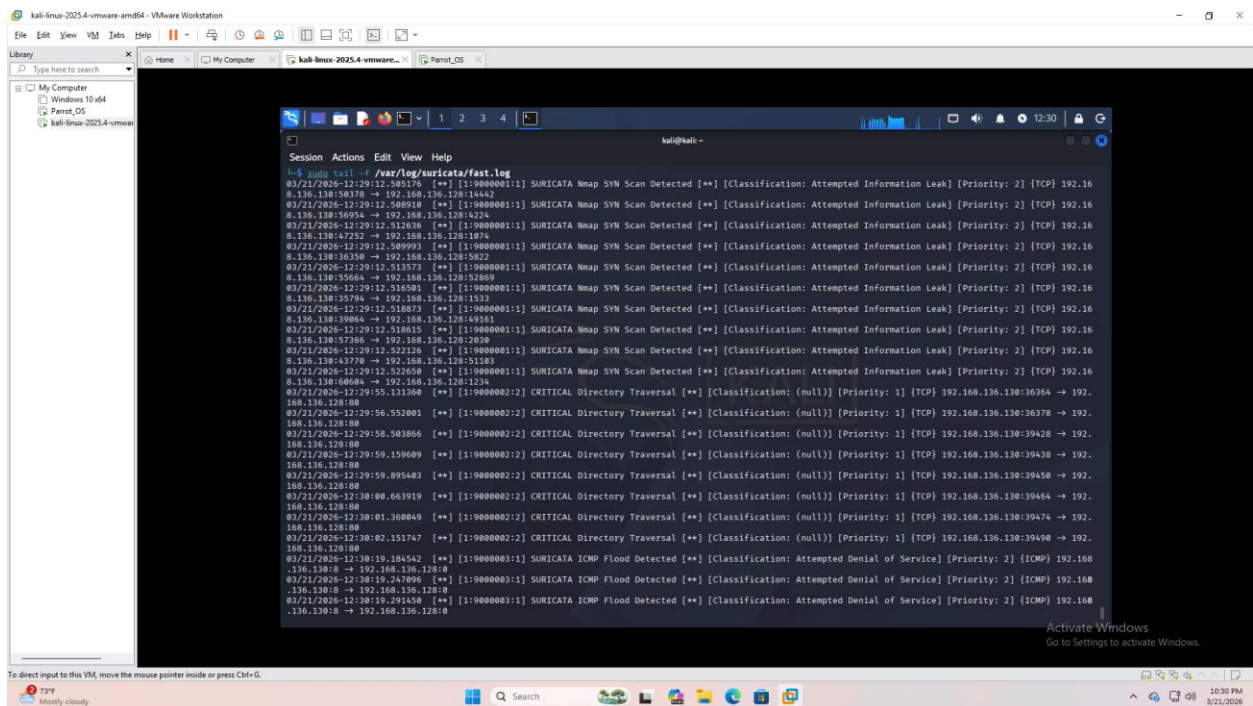
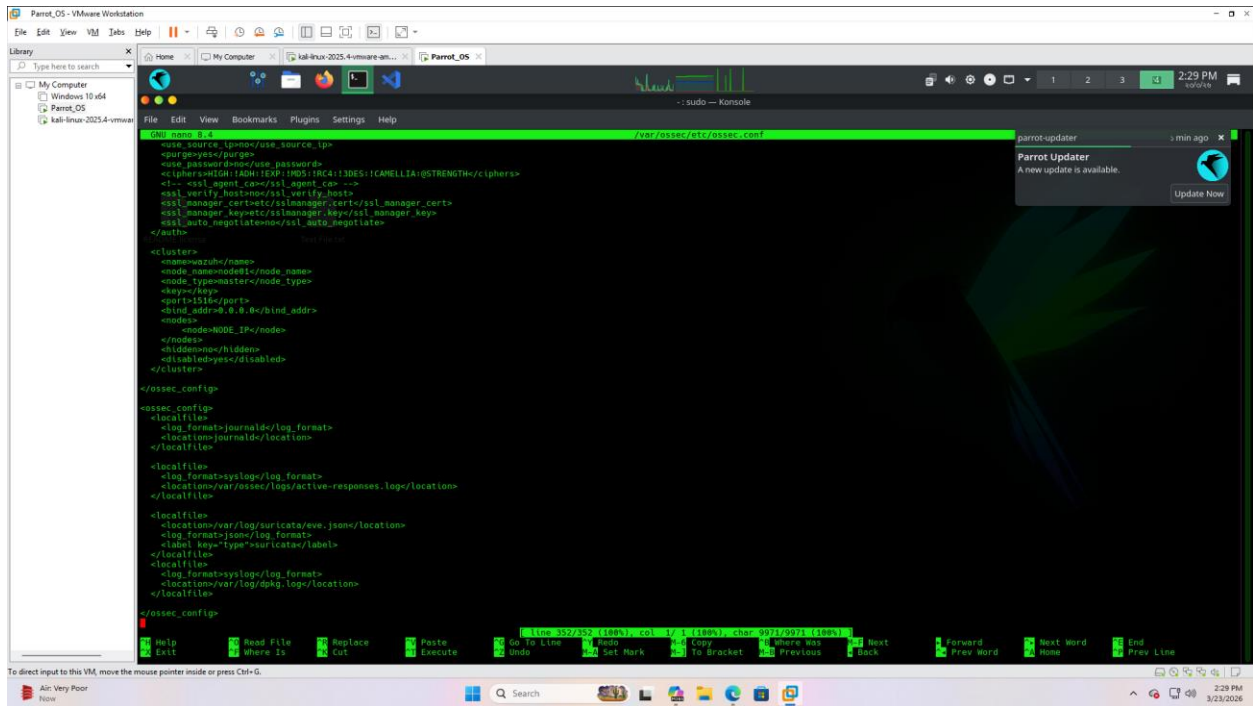
On Kali, I edited:

```
sudo nano /var/ossec/etc/ossec.conf
```

I added:

```
<localfile>  
  <location>/var/log/suricata/eve.json</location>  
  <log_format>json</log_format>  
  <label key="type">suricata</label>  
</localfile>
```


Screenshot



3.2 Restart Agent / Manager

On Kali

```
sudo systemctl restart wazuh-agent  
sudo systemctl status wazuh-agent
```

On Parrot

```
sudo systemctl restart wazuh-manager  
sudo systemctl status wazuh-manager
```

What to write

After adding Suricata's JSON log to the Wazuh agent configuration, the agent was restarted so new log monitoring settings would take effect. The Wazuh manager was also checked to ensure the SIEM side remained healthy

3.3 Verify Integration

What I did

I generated test traffic again from Kali:

```
nmap ip
```

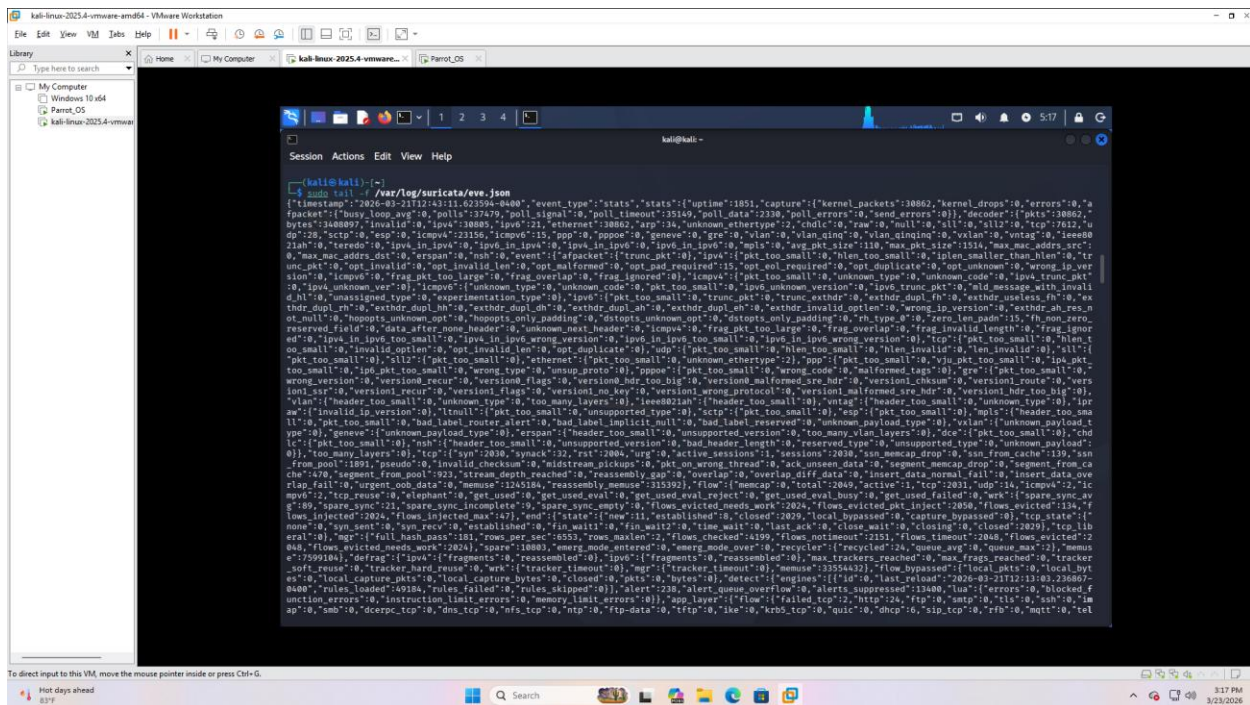
Then I confirmed eve.json was receiving alerts:

```
sudo tail -f /var/log/suricata/eve.json
```

What to write

This step verified that Suricata events were still being generated normally after Wazuh log collection was enabled.

ScreenShot:



3.4 Check Wazuh Dashboard

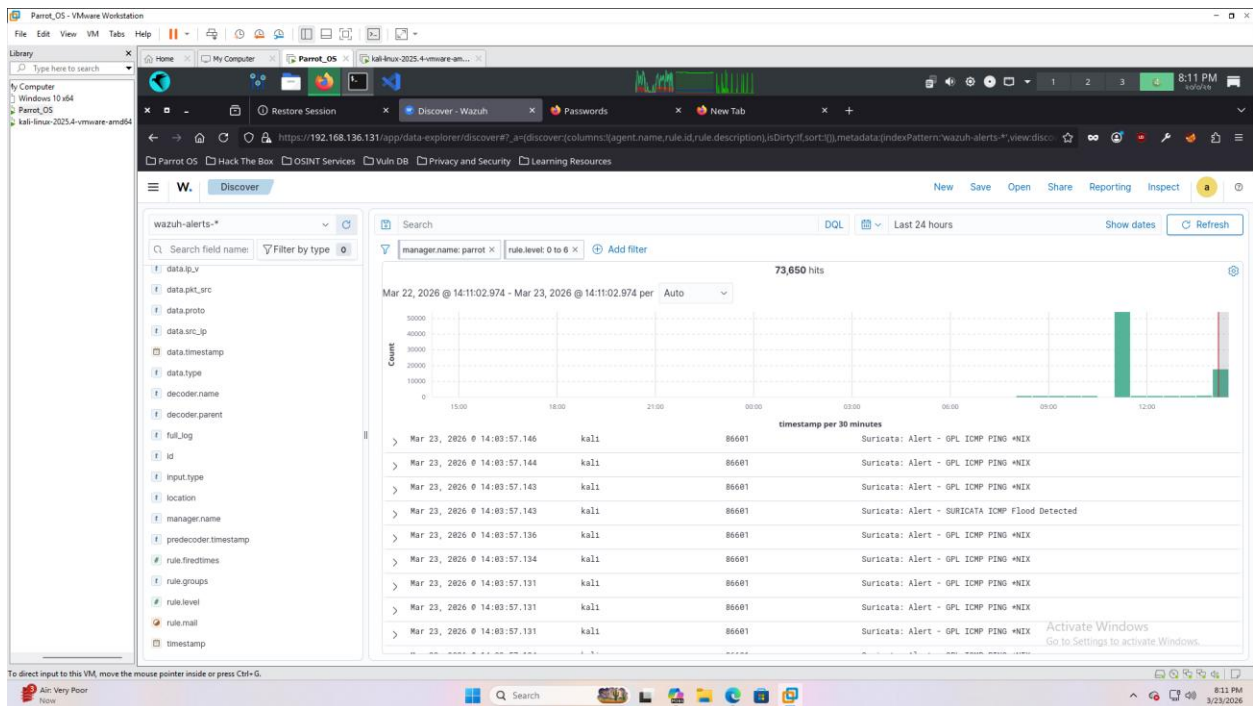
What I did

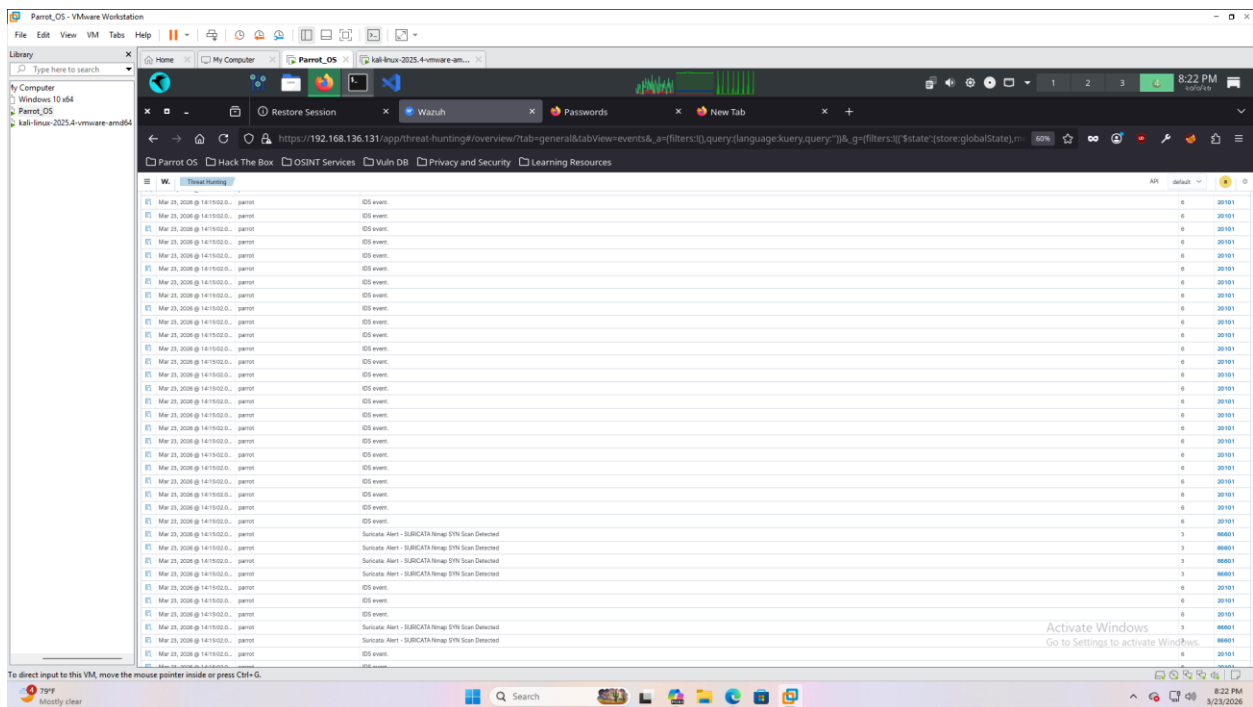
On the Parrot Wazuh dashboard, I opened Security Events / Threat Hunting and searched for Suricata-related alerts.

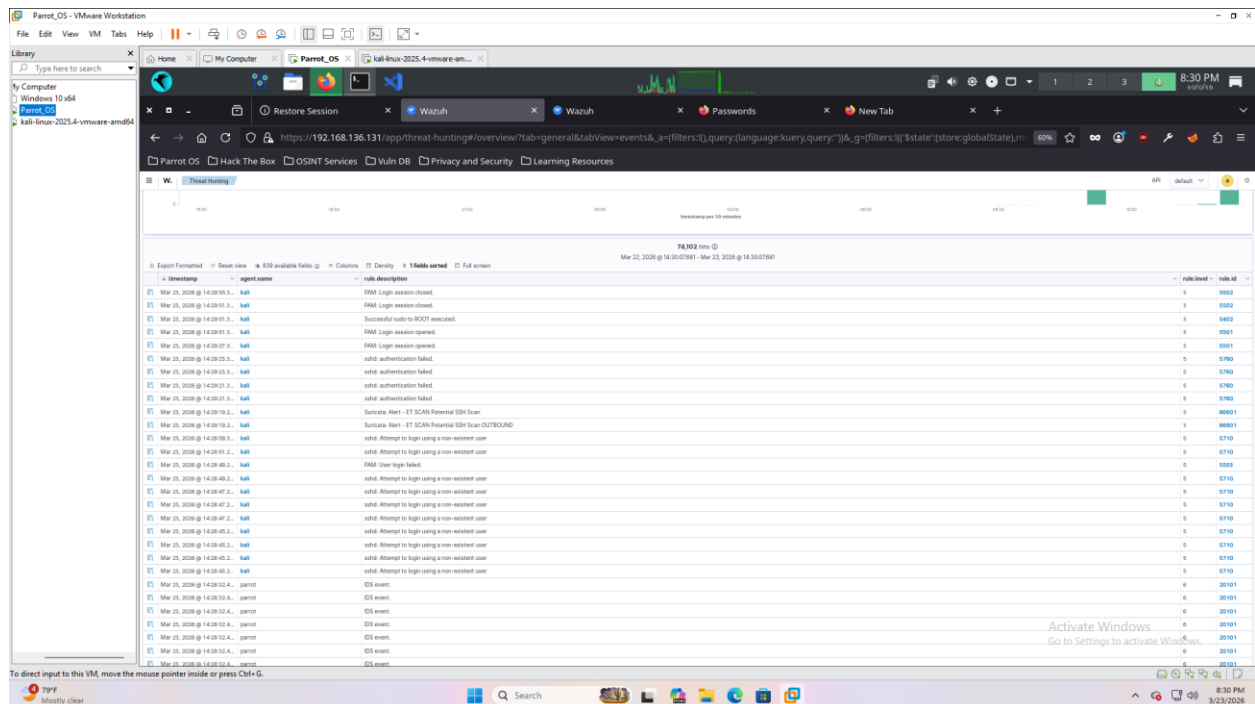
What to write

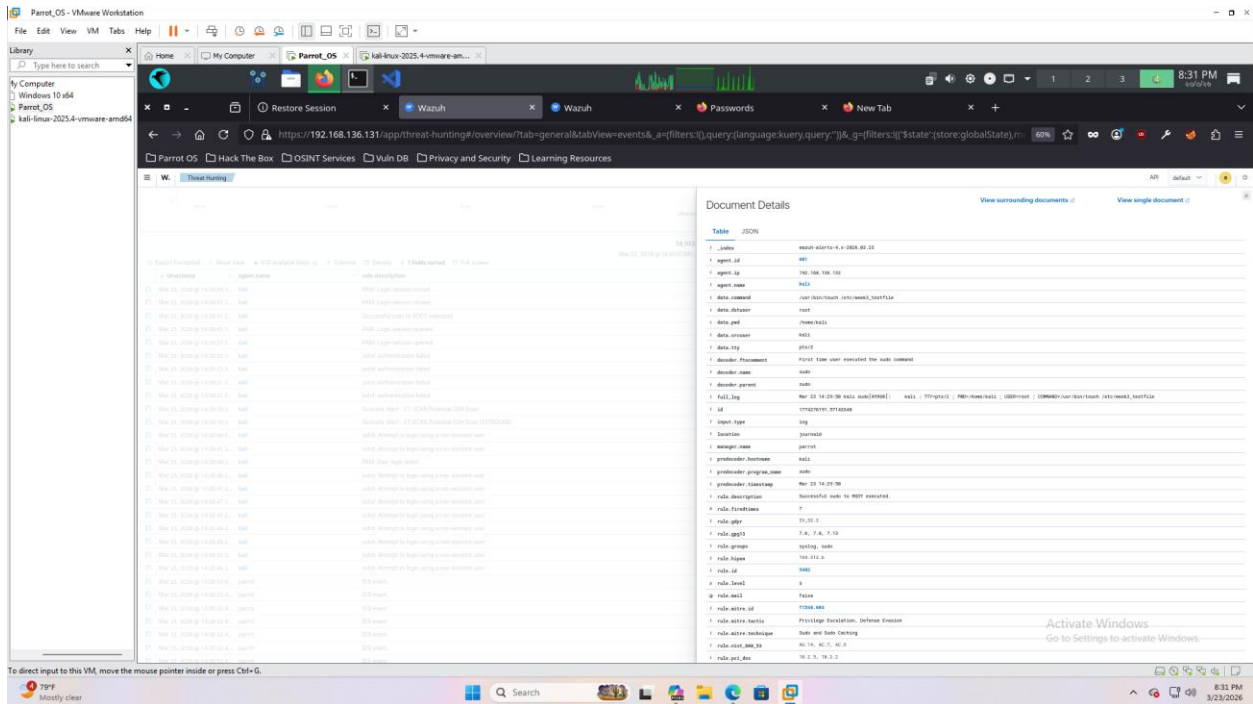
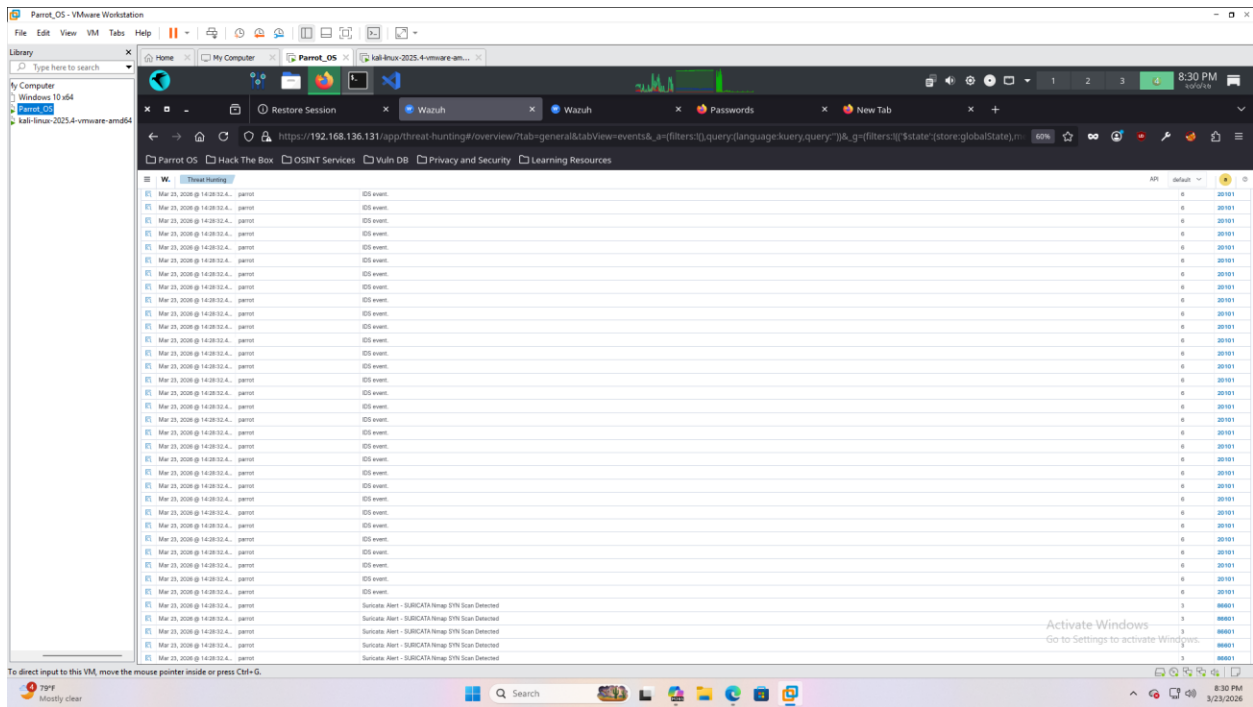
The dashboard displayed parsed Suricata network alerts, confirming successful SIEM integration. This is the data flow architecture, where `eve.json` feeds Wazuh ingestion and then appears at the Wazuh manager/dashboard layer.

Screenshots









3.6 Event Flow Understanding

What to write exactly

The full alert flow in this lab was:

1. Network traffic was generated on the host-only interface.
2. Suricata inspected the packets through the af-packet capture mechanism.
3. If the traffic matched a signature or threshold condition, Suricata created an alert.
4. The alert was written to `fast.log` and `eve.json`.
5. The Wazuh agent /manager on Kali Linux/parrot os read `eve.json`.
6. The agent forwarded the event to the Wazuh manager on Parrot.
7. Wazuh decoded and indexed the event.
8. The alert appeared in the Wazuh dashboard for search and analysis.

4. GeoIP Blocking & Firewall

4.1 Download Country Blocklist

Commands

```
mkdir -p ~/geoip-block  
cd ~/geoip-block  
wget https://www.ipdeny.com/ipblocks/data/countries/cn.zone
```

What to write

I downloaded country-based IP blocklists to use for perimeter filtering. This step matches the final requirement in the Week 3 brief, which asks for GeoIP-based firewall blocking to reduce attack surface.

Screenshot

```
(kali@kali)~$ mkdir -p ~/geoip-block
cd ~/geoip-block
wget https://www.ipdeny.com/ipblocks/data/countries/cn.zone
--2026-03-23 15:47:04-- https://www.ipdeny.com/ipblocks/data/countries/cn.zone
Resolving www.ipdeny.com (www.ipdeny.com)... 51.15.12.186
Connecting to www.ipdeny.com (www.ipdeny.com)[51.15.12.186]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 137026 (134K) [text/plain]
Saving to: 'cn.zone'
cn.zone 100%[=====] 133.81K 370KB/s in 0.4s
2026-03-23 15:47:06 (370 KB/s) - 'cn.zone' saved [137026/137026]
```

```
(kali@kali)~$ mkdir -p ~/geoip-block
cd ~/geoip-block
wget https://www.ipdeny.com/ipblocks/data/countries/cn.zone
--2026-03-23 07:21:03-- https://www.ipdeny.com/ipblocks/data/countries/cn.zone
Resolving www.ipdeny.com (www.ipdeny.com)... 51.15.12.186
Connecting to www.ipdeny.com (www.ipdeny.com)[51.15.12.186]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 137026 (134K) [text/plain]
Saving to: 'cn.zone'
cn.zone 100%[=====] 133.81K 364KB/s in 0.4s
2026-03-23 07:21:04 (364 KB/s) - 'cn.zone' saved [137026/137026]
```

4.2 Create IP Set

Commands

```
sudo ipset create block_cn hash:net
```

4.3 Add IPs to Set

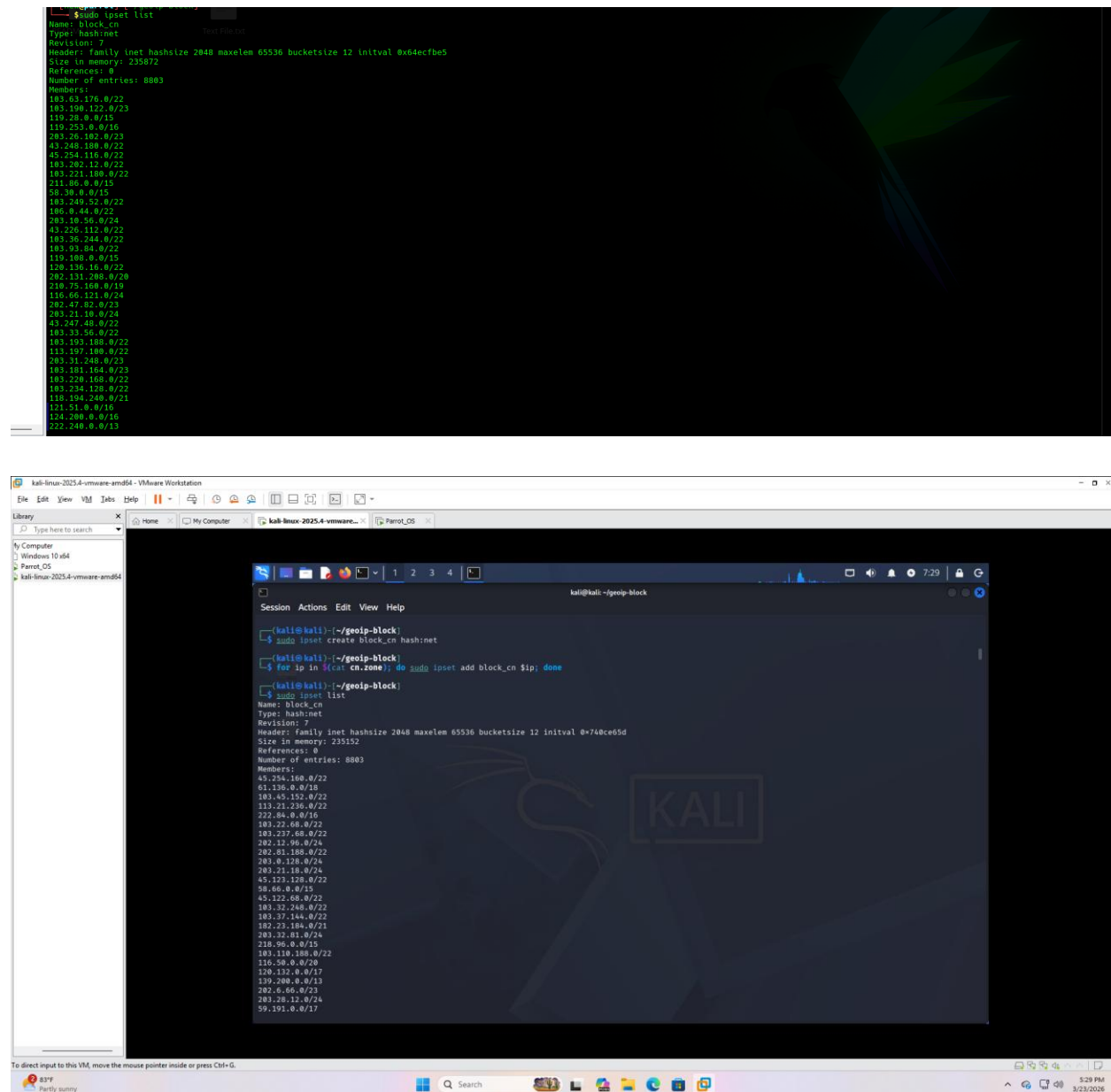
Commands

```
for ip in $(cat cn.zone); do sudo ipset add block_cn $ip;
done
```

Verification

```
sudo ipset list
```

Screenshot



4.4 Apply Firewall Rule

Commands

```
sudo iptables -A INPUT -m set --match-set block_cn src -j DROP
```


4.6 Verify Rules

Command

```
sudo iptables -L INPUT -n --line-numbers
```

What to write

The INPUT chain confirmed that packets from the selected country IP sets would be dropped. This proved that the GeoIP block policy was loaded into the firewall successfully.

4.7 Test Blocking

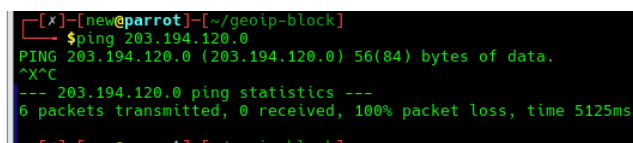
What to write

Practical GeoIP testing is difficult in a closed lab unless traffic is actually sourced from those country ranges. Therefore, I verified the block in three ways:

1. I confirmed the .zone files contained country CIDR ranges.
2. I verified those ranges were loaded into the relevant IP sets.
3. I verified the firewall INPUT chain referenced those IP sets with DROP.

This demonstrated that if traffic originated from those IP ranges, it would be blocked by the firewall.

Screenshot



```
[x]-[new@parrot]-[~/geolp-block]
$ping 203.194.120.0
PING 203.194.120.0 (203.194.120.0) 56(84) bytes of data:
^X^C
--- 203.194.120.0 ping statistics ---
6 packets transmitted, 0 received, 100% packet loss, time 5125ms
```

```
(kali@kali)~/geoiip-block
$ ping 203.194.120.0
PING 203.194.120.0 (203.194.120.0) 56(84) bytes of data.
^C
— 203.194.120.0 ping statistics —
3 packets transmitted, 0 received, 100% packet loss, time 2046ms

(kali@kali)~/geoiip-block
$
```

4.8 Key Concepts

What to write

- **Wazuh** is host-based monitoring
- **Suricata** is network-based monitoring
- **IDS mode** detects and logs traffic but does not block it
- **IPS mode** can actively block traffic
- **fast.log** is compact and human-readable
- **eve.json** is structured and used for SIEM ingestion
- **Threshold rules** are necessary for flood/scan behavior
- **GeoIP blocking** reduces exposure but is not a complete security solution

Beginner Reflection and Learning Notes

From a beginner perspective, this was important because it introduced the difference between host-based monitoring and network-based monitoring in a practical way. Before this lab, it was easy to think that a SIEM alone was enough to detect all security events. After completing the exercises, it became clear that different tools cover different parts of the environment.

One of the biggest lessons from this week was that security tools can appear to be working even when they are not fully functional. For example, Suricata may show as active and running, but if the wrong interface is selected or no rules are loaded, it may inspect no useful traffic and generate no meaningful

alerts. This highlighted the importance of end-to-end verification rather than relying only on service status.

Another useful lesson was understanding that not all blocking methods are the same. GeoIP blocking with `ipset` and `iptables` works at the network level, while domain blocking through the `/etc/hosts` file works only at the local machine's name-resolution level. Learning this distinction helped build a clearer understanding of how defensive controls operate at different layers.

Overall, this was a practical introduction to how SOC analysts combine multiple data sources to improve visibility. Even though some configuration and troubleshooting steps were challenging, they helped build a stronger foundation in IDS concepts, alert validation, log forwarding, and firewall-based restriction technique

5. Conclusion

This lab successfully introduced network-based detection into the existing Wazuh SIEM environment through Suricata integration. Following the same overall structure as the sample report, the work covered installation, interface selection, Suricata configuration, rule writing, Wazuh ingestion, event correlation, and GeoIP-based firewall blocking.

Suricata was installed and configured to monitor the NAT lab interface, ensuring visibility into attack traffic generated during testing. Custom rules were written for three threat scenarios: Nmap SYN scan activity, HTTP directory traversal attempts, and ICMP flood behavior. These rules were validated through live testing and verified in Suricata's logs.

Suricata telemetry was then integrated into Wazuh by monitoring `eve.json` through the Wazuh agent. This enabled network alerts to appear alongside host-based alerts in the Wazuh dashboard, which supports stronger

correlation and investigation. Finally, GeoIP-based firewall controls were applied using `ipset` and `iptables`, and domain blocking was demonstrated using the `hosts` file.

Overall, the lab showed why both tools are needed together: Wazuh provides host visibility, Suricata provides network visibility, and the combination significantly reduces blind spots in a SOC monitoring environment.